arXiv:2603.00195v1 [cs.CR] 27 Feb 2026

Formal Analysis and Supply Chain Security for Agentic AI Skills

Varun Pratap Bhardwaj
varun.pratap.bhardwaj@gmail.com
Independent Research
Solution Architect
India

Abstract

The rapid proliferation of agentic AI skill ecosystems—exemplified by OpenClaw (228,000 GitHub stars) and Anthropic Agent Skills (75,600 stars)—has introduced a critical supply chain attack surface. The ClawHavoc campaign (January–February 2026) infiltrated over 1,200 malicious skills into the OpenClaw marketplace, while MalTool catalogued 6,487 malicious tools that evade conventional detection. In response, twelve reactive security tools emerged, yet *all* rely on heuristic methods that provide no formal guarantees.

We present **SkillFortify**, the first formal analysis framework for agent skill supply chains, with six contributions: (1) the *DY-Skill* attacker model, a Dolev–Yao adaptation to the five-phase skill lifecycle with a maximality proof; (2) a sound static analysis framework grounded in abstract interpretation; (3) capability-based sandboxing with a confinement proof; (4) an Agent Dependency Graph with SAT-based resolution and lockfile semantics; (5) a trust score algebra with formal monotonicity; and (6) SkillFortifyBench, a 540-skill benchmark. SkillFortify achieves 96.95% F1 (95% CI: [95.1%, 98.4%]) with 100% precision and 0% false positive rate on 540 skills, while SAT-based resolution handles 1,000-node graphs in under 100 ms.

Keywords

supply chain security, agent skills, formal analysis, capability-based security, abstract interpretation, Dolev–Yao model, AI safety

1 Introduction

On January 27, 2026, security researchers at Ethiack disclosed CVE-2026-25253 [21], a remote code execution vulnerability in the OpenClaw agent skill runtime—the first Common Vulnerabilities and Exposures identifier assigned to an agentic AI system. Within days, the *ClawHavoc* campaign [68] exploited this and related weaknesses to infiltrate over 1,200 malicious skills into the OpenClaw marketplace, deploying the AMOS credential stealer to developer workstations. Concurrently, MalTool benchmarked 6,487 malicious tools targeting LLM-based agents [63], demonstrating that VirusTotal fails to detect the majority of agent-targeted malware. A large-scale empirical analysis of 42,447 agent skills found that 26.1% exhibit at least one security vulnerability [54].

These incidents are not isolated. They reflect a structural vulnerability in the emerging agent skill ecosystem: *developers install and execute third-party skills with implicit trust and no formal analysis.* The agent skill supply chain—spanning authorship, registry publication, developer installation, runtime execution, and state persistence—reproduces the trust pathologies of traditional package ecosystems (npm, PyPI, crates.io) but with amplified risk, because agent skills execute with broad system privileges in the context of a powerful language model.

1.1 The Defense Gap

The security community responded rapidly. Within thirty days of the ClawHavoc disclosure, twelve reactive security tools appeared on GitHub [68]. The most prominent include:

- **Snyk agent-scan** (~1,500 stars): Acquired Invariant Labs and applies LLM-as-judge combined with heuristic rules to scan agent configurations [60].
- **Cisco skill-scanner** (994 stars): Employs YARA pattern matching and signature-based detection for known malicious skill patterns [12].
- **ToolShield** [62]: A heuristic defense achieving a 30% reduction in attack success rate through behavioral heuristics.

Despite their utility, *every* existing tool shares a fundamental limitation: they are heuristic. They detect known patterns but cannot prove the absence of malicious behavior. Cisco's own documentation for their skill-scanner states explicitly: "no findings does not mean no risk" [12]. The closest academic work toward formal guarantees is a four-page ICSE-NIER vision paper proposing Alloy-based modeling of tool safety [48], which provides neither an implementation nor empirical evaluation.

This gap is the central problem we address: *the agent skill ecosystem has attack frameworks but no defense frameworks with formal guarantees.*

1.2 Our Approach

We present **SkillFortify**, the first formal analysis framework for agent skill supply chains. SkillFortify provides sound static analysis with mathematical guarantees—not heuristic scanning—that skills cannot exceed their declared capabilities. Where existing tools answer *"did we detect a known bad pattern?"*, SkillFortify answers *"can this skill access resources beyond what it declares?"* and provides a machine-checkable certificate when the answer is no.

Our approach draws on four decades of formal methods research, adapting established foundations to the novel domain of agent skill security: the Dolev–Yao attacker model [19] for supply chain threat modeling, abstract interpretation [14] for sound static analysis, capability-based security [17, 44] for sandboxing, and SAT-based dependency resolution [61] for supply chain management.

1.3 Contributions

This paper makes six contributions:

(1) **DY-Skill attacker model** (Section 3). We introduce a novel adaptation of the Dolev–Yao formalism [19] to the five-phase agent skill supply chain. The DY-Skill attacker can intercept, inject, synthesize, decompose, and replay skill messages. We prove that this attacker is *maximal*: any symbolic attacker on the supply chain can be simulated by a DY-Skill trace (*Theorem 3.6*).
(2) **Static analysis with soundness guarantees** (Section 4). We develop a three-phase static analysis framework grounded in abstract interpretation over a four-element capability lattice ($\{\text{NONE}, \text{READ}, \text{WRITE}, \text{ADMIN}\}, \sqsubseteq$). We prove that if our analysis reports a skill as safe, then its concrete execution cannot access resources beyond its declared capabilities (*Theorem 4.9*).
(3) **Capability-based sandboxing with confinement proof** (Section 5). We formalize a capability model for agent skills based on the object-capability discipline [41, 44]. We prove that in a capability-safe execution environment, no skill can acquire authority exceeding the transitive closure of its declared capability set—the *no authority amplification* property (*Theorem 5.6*).
(4) **Agent Dependency Graph with SAT-based resolution** (Section 6). We define the Agent Dependency Graph $\mathcal{G} = (\mathcal{S}, V, D, C, \text{Cap})$ extending classical package dependency models [42, 61] with per-skill capability constraints. We encode the resolution problem as a SAT instance with security bounds and prove that any satisfying assignment corresponds to a *secure installation* with a deterministic lockfile (*Theorem 6.10*).
(5) **Trust score algebra with formal propagation** (Section 7). We introduce a multi-signal trust scoring system with provenance, behavioral, community, and historical dimensions. Trust propagates multiplicatively through dependency chains and decays exponentially for unmaintained skills. We prove the *monotonicity* property: additional positive evidence never reduces a skill's trust score (*Theorem 7.12*).
(6) **SkillFortifyBench and empirical evaluation** (Section 9). We construct SkillFortifyBench, a 540-skill benchmark comprising 270 malicious and 270 benign skills drawn from MalTool [63], ClawHavoc [68], and curated clean sources. SkillFortify achieves an F1 score of 96.95% with 100% precision and 0% false positive rate on benign skills. SAT-based dependency resolution handles 1,000-node graphs in under 100 ms. We evaluate on SkillFortifyBench (540 skills) and position our results against reported metrics from existing tools, demonstrating that combined pattern matching and information flow analysis detects attack patterns invisible to pure pattern matching alone.

1.4 Paper Organization

Section 2 surveys the agent skill ecosystem, attack landscape, existing defenses, and formal methods foundations. Section 3 presents the DY-Skill attacker model and threat taxonomy. Section 4 develops the static analysis framework. Section 5 formalizes capability-based sandboxing. Section 6 defines the Agent Dependency Graph and lockfile semantics. Section 7 introduces the trust score algebra. Section 8 describes the SkillFortify implementation. Section 9 presents the empirical evaluation. Section 10 positions our contributions against related work. Section 11 discusses limitations and future directions, and Section 12 concludes. Full proofs appear in Section A, benchmark details in Section B, and the lockfile schema in Section C.

2 Background

This section surveys the agent skill ecosystem (§2.1), the attack landscape (§2.2), existing defense tools (§2.3), traditional supply chain security foundations (§2.4), and the formal methods we build upon (§2.5). A dedicated comparison with related work appears in Section 10.

2.1 Agent Skill Ecosystems

The year 2025–2026 witnessed explosive growth in agent skill platforms. An *agent skill* is a third-party extension—typically a code module, configuration file, or API endpoint—that augments an LLM-based agent with domain-specific capabilities such as web search, file manipulation, database querying, or interaction with external services.

Three ecosystems dominate the landscape:

OpenClaw. With 228,000 GitHub stars as of February 2026, OpenClaw is the largest open-source agent framework. Its marketplace operates on an *install-and-run* trust model: developers discover skills through a web interface or CLI, install them with a single command, and skills execute with the agent's full system privileges. No formal review, signing, or capability declaration is required for publication.

Anthropic Agent Skills. Anthropic's official skill repository (75,600 stars) provides curated skills for Claude Code and Claude Desktop. Skills are defined as YAML/Markdown files in `.claude/skills/` directories and may include shell commands, code blocks, and instructions. While Anthropic maintains a moderation pipeline, no formal capability model constrains skill behavior at runtime.

Model Context Protocol (MCP).. The Model Context Protocol [1] standardizes how agents interact with external data sources and tools through JSON-RPC servers. MCP servers declare tool schemas (input/output types) but not *capability requirements*—a server that claims to "search files" may also execute arbitrary shell commands, and the protocol provides no mechanism to detect or prevent this.

All three ecosystems share a common structural weakness: the absence of a *formal capability model* that constrains what a skill can do to what it declares. This is the gap our work fills.

2.2 Attack Landscape

Five incidents and studies define the attack landscape as of February 2026. Early work on LLM platform plugin security [32] identified many of the attack vectors that have since materialized at scale, and recent systematic analyses of the Model Context Protocol [30] confirm that these vulnerabilities persist in modern agent architectures:

CVE-2026-25253 (January 27, 2026). The first CVE assigned to an agentic AI system. Ethiack demonstrated remote code execution in the OpenClaw skill runtime through a crafted skill package. The proof-of-concept repository received 75 stars within days [21].

ClawHavoc Campaign (Late January–Early February 2026). A coordinated supply chain attack documented in the "SoK: Agentic Skills in the Wild" systematization [68]. Attackers published over 1,200 malicious skills to the OpenClaw marketplace, deploying the AMOS credential stealer. The paper identifies seven distinct skill design patterns exploited by the campaign and categorizes the full attack lifecycle.

MalTool (February 12, 2026). Researchers from Dawn Song's group catalogued 6,487 malicious tools targeting LLM-based agents [63]. Their benchmark demonstrated that VirusTotal—the de facto standard for malware detection—fails to identify the majority of agent-targeted malware, underscoring the inadequacy of signature-based approaches for this domain. Complementary MCP-specific attack research—MCP-ITP [39] (automated implicit tool poisoning) and MCPTox [55] (benchmarking tool poisoning on real-world MCP servers)—demonstrates that protocol-level vulnerabilities compound the skill-level threat.

Agent Skills in the Wild (January 2026). A large-scale empirical study scanned 42,447 agent skills across multiple registries [54]. Key findings: 26.1% of skills exhibit at least one security vulnerability, 14 distinct vulnerability patterns were identified, and their SkillScan tool achieves 86.7% precision—leaving a 13.3% gap that formal methods can address.

Malicious Agent Skills in the Wild (February 2026). A large-scale registry scan covering 98,380 skills across multiple agent platforms identified 157 confirmed malicious entries [37], independently corroborating the vulnerability rates reported by Park et al. [54] on a substantially larger corpus.

Multi-agent data leakage. OMNI-LEAK [13] demonstrates that multi-agent architectures create implicit information channels through which sensitive data can leak across trust boundaries—even when individual agents appear well-isolated. This motivates our information flow analysis component (Section 4).

Agent reliability. Chen et al. [10] provide empirical evidence that LLM-based agents exhibit stochastic path deviation even on well-defined tasks, confirming that the combination of unreliable agents and unverified skills creates compounding risk.

Malice in Agentland (October 2025). A study demonstrating that poisoning just 2% of an agent's execution trace is sufficient to achieve 80% attack success in multi-agent systems [16]. This validates the threat model of a supply chain attacker who need only compromise a single skill in a dependency chain to affect an entire agent pipeline.

2.3 Existing Defenses

All existing defenses are heuristic. We survey the most prominent.

Snyk agent-scan (~1,500 stars). The most well-funded entrant, following Snyk's acquisition of Invariant Labs in early 2026 [60].

Table 1: Comparison of agent skill security tools. SkillFortify is the first to provide formal guarantees (soundness, confinement, resolution correctness).

Tool	Formal Model	Dep. Graph	Lock-file	Trust Score	SBoM
Snyk agent-scan	✗	✗	✗	✗	✗
Cisco skill-scanner	✗	✗	✗	✗	✗
ToolShield	✗	✗	✗	✗	✗
ICSE-NIER '26	Partial	✗	✗	✗	✗
mcp-audit	✗	✗	Partial	✗	Partial
MCPShield	✗	✗	Partial	✗	✗
SkillFortify	✓	✓	✓	✓	✓

Applies LLM-as-judge combined with hand-crafted rules to evaluate agent configurations and skill manifests. Strengths: deep CI/CD integration, developer-friendly UX. Limitation: heuristic rules cannot prove absence of malicious behavior; LLM judges are themselves susceptible to adversarial inputs [63].

Cisco skill-scanner (994 stars). Employs YARA pattern matching and regex-based signature detection for known malicious skill patterns [12]. The documentation explicitly states: "no findings does not mean no risk." While effective for known patterns, the approach cannot generalize to novel attack vectors.

ToolShield [62]. A heuristic defense from the Dawn Song and Bo Li groups achieving a 30% reduction in attack success rate. Uses behavioral heuristics derived from the MalTool benchmark. Like all heuristic approaches, it cannot provide formal guarantees about the completeness of its detection.

MCPShield [43]. Implements `mcp.lock.json` with SHA-512 content hashes for tamper detection of MCP server configurations. While MCPShield's lockfile provides *integrity verification* (detecting unauthorized modifications), it does not perform behavioral analysis, dependency resolution, or capability verification. SkillFortify's lockfile (Section C) extends beyond hash-based tamper detection: each locked entry records the SAT-based resolution result with version constraints *and* security bounds, the computed trust score, and the formal analysis status—making the lockfile a formally-proven satisfying assignment, not merely an integrity manifest.

Towards Verifiably Safe Tool Use [48]. The closest academic work to our approach: a four-page ICSE-NIER '26 vision paper proposing Alloy-based formal modeling of tool safety properties. While the paper correctly identifies the need for formal methods, it provides neither a working implementation, a threat model, nor empirical evaluation. Our work realizes the vision this paper articulates, extending it with a complete formal framework, five theorems, a working tool, and a 540-skill benchmark.

Summary. Table 1 compares existing defenses. SkillFortify is the first to provide formal guarantees—soundness, confinement, and resolution correctness—while also implementing a practical tool.

2.4 Traditional Supply Chain Security

Agent skill security inherits lessons—and gaps—from four decades of software supply chain research.

Package manager security. Duan et al. [20] systematized supply chain attacks on npm, PyPI, and RubyGems, identifying 339 malicious packages through metadata, static, and dynamic analysis. Ohm et al. [49] catalogued "backstabber" packages, and Ladisa et al. [36] proposed a comprehensive taxonomy of open-source supply chain attacks. Gibb et al. [24] introduced a *package calculus* providing a unified formalism across ecosystem boundaries. We extend this line of work to agent skills, where the threat surface is amplified by LLM integration and the absence of established security norms.

SLSA Framework. Supply-chain Levels for Software Artifacts [26] defines four graduated trust levels (L1–L4) based on build provenance and integrity guarantees. Our trust score algebra (Section 7) adapts the SLSA graduation model to agent skills, mapping L1 (basic provenance) through L4 (formal verification) to concrete, measurable criteria.

Sigstore and code signing. Sigstore [38] provides keyless code signing for open-source packages. While we do not implement signing in this work (deferred to V2), our trust score model explicitly incorporates signing status as a provenance signal, and our lockfile format includes fields for cryptographic attestations.

CycloneDX and SBoM.. The CycloneDX specification [51] standardizes Software Bills of Materials, recently extended to AI components (AI-BOM) [52]. SkillFortify generates Agent Skill Bills of Materials (ASBoM) in CycloneDX format, enabling integration with existing enterprise compliance pipelines.

NIST and regulatory frameworks. The NIST AI Risk Management Framework [47] and the EU AI Act [22] both address third-party AI component risks. The EU AI Act Article 17 requires providers of high-risk AI systems to implement supply chain risk management. SkillFortify's formal verification, lockfile semantics, and ASBoM output provide concrete mechanisms for regulatory compliance.

2.5 Formal Methods Foundations

Our framework synthesizes five established formal methods traditions.

Dolev–Yao model. Dolev and Yao [19] introduced the standard symbolic attacker model for security protocol analysis. The attacker controls the network and can intercept, inject, synthesize (construct new messages from known components), and decompose (extract components from messages) messages. Cervesato [9] proved that the DY intruder is the *most powerful* attacker in the symbolic model—any attack achievable by any symbolic adversary can be simulated by a DY trace. We adapt this model to the agent skill supply chain (Section 3), introducing the DY-Skill attacker with a fifth operation (replay) and a "perfect sandbox assumption" analogous to the origina[illegible]rfect cryptography assumption.

Abstract interpretation. Cousot and Cousot [14] introduced abstract interpretation as a theory of sound approximation of program semantics. A concrete domain $(C, \subseteq)$ is connected to an abstract domain $(A, \sqsubseteq)$ through a Galois connection (α, γ), and abstract fixpoint computation provides a sound over-approximation of concrete behavior. The Astrée analyzer [15] demonstrated that abstract interpretation can prove absence of runtime errors in safety-critical avionics software. We apply this framework to agent skill analysis (Section 4), using a four-element capability lattice as our abstract domain.

Capability security. Dennis and Van Horn [17] introduced capabilities as unforgeable references coupling resource designations with access rights. Miller [44] extended this to the object-capability (ocap) model, establishing the Principle of Least Authority (POLA). Maffeis, Mitchell, and Taly [41] proved the capability safety theorem: in a capability-safe language, the reachability graph of object references is the *only* mechanism for authority propagation, guaranteeing that untrusted code cannot exceed the authority explicitly delegated to it. Our sandboxing model (Section 5) instantiates these results for agent skills.

SAT-based dependency resolution. Mancinelli et al. [42] proved that package dependency resolution is NP-complete. Tucker et al. [61] introduced OPIUM, which encodes dependency resolution as a SAT problem amenable to efficient solving via CDCL (Conflict-Driven Clause Learning). Di Cosmo and Vouillon [18] verified graph transformations for co-installability analysis in Coq. We extend this line of work (Section 6) by adding per-skill capability constraints to the SAT encoding, producing dependency resolutions that satisfy both version co[illegible]aints *and* security bounds.

Trust management. Blaze, Feigenbaum, and Lacy [7] introduced PolicyMaker, establishing the assertion monotonicity property: adding credentials never reduces compliance. The KeyNote system [6] extended this with multi-valued compliance and delegation filters. Our trust score algebra (Section 7) draws on these foundations, implementing monotonic trust propagation through dependency chains with exponential decay for unmaintained skills.

3 Formal Threat Model

We develop a formal threat model for the agent skill supply chain by (1) defining the supply chain as a structured tuple of interacting entities, (2) enumerating attack classes with phase-applicability mappings, and (3) introducing the *DY-Skill* attacker model—a novel adaptation of the Dolev–Yao symbolic attacker [19] to the agent skill domain.

3.1 Agent Skill Supply Chain

An *agent skill supply chain* is a tuple $\mathcal{SC} = (\mathcal{A}, \mathcal{R}, \mathcal{D}, \mathcal{E})$ whose components are defined as follows.

Definition 3.1 (Agent Skill Supply Chain)**.** Let

- $\mathcal{A}$ be a finite set of *skill authors* who produce skill packages,
- $\mathcal{R}$ be a finite set of *registries* that store and distribute skill packages,
- $\mathcal{D}$ be a finite set of *developers* (or agents) that install and configure skills, and
- $\mathcal{E}$ be a finite set of *execution environments* in which skills run.

A skill traverses $\mathcal{SC}$ through a five-phase lifecycle $\pi = \langle$INSTALL, LOAD, CONFIGURE, EXECUTE, PERSIST$\rangle$, formally an ordered sequence of phases $\pi_1 \prec \pi_2 \prec \cdots \prec \pi_5$.

Each phase exposes a distinct attack surface:

Phase 1: INSTALL. A skill package is fetched from a registry $r \in \mathcal{R}$ and placed in the local environment. The developer trusts the registry to serve the requested skill with the correct name and version. *Attack surfaces*: name confusion (typosquatting, namespace squatting), dependency confusion, registry compromise.

Phase 2: LOAD. The skill manifest is parsed, dependencies resolved, and code modules imported into the agent's runtime. Skill descriptions and metadata are presented to the agent's language model for tool selection. *Attack surfaces*: prompt injection via metadata, code injection in initialization hooks.

Phase 3: CONFIGURE. The skill is parameterized for a specific agent and environment. Configuration may include API keys, endpoint URLs, and access scopes. *Attack surfaces*: over-permissioning, configuration template injection, capability escalation.

Phase 4: EXECUTE. The skill runs within the agent's execution context, processing user requests and producing tool outputs. This is the phase with the broadest authority: skills may invoke subtools, read files, and make network requests. *Attack surfaces*: data exfiltration, privilege escalation, return-value prompt injection.

Phase 5: PERSIST. The skill writes state, logs, or artifacts to durable storage. Persistence operations may leak data to attacker-readable locations. *Attack surfaces*: data exfiltration through logs/shared storage, state poisoning.

A key observation is that attacks at phase π_i can propagate forward to phases $\pi_{i+1}, \ldots, \pi_5$. For example, a typosquatting attack at INSTALL enables arbitrary code execution at EXECUTE. We call this the *forward propagation property* of supply chain attacks.

3.2 Attack Taxonomy

We identify six formal attack classes derived from empirical analysis of the ClawHavoc campaign [68] (1,200+ malicious skills), the MalTool benchmark [63] (6,487 malicious tools), and the "Agent Skills in the Wild" survey [54] (42,447 skills).

Definition 3.2 (Attack Classes)**.** Let $C = \{c_1, \ldots, c_6\}$ be the set of attack classes, and let $\Phi\colon C \to 2^{\{\pi_1,\ldots,\pi_5\}}$ map each class to its applicable supply chain phases:

(1) **Data Exfiltration** (c_1): Unauthorized extraction of sensitive data (API keys, conversation history, environment variables) to an attacker-controlled endpoint. $\Phi(c_1) = \{$EXECUTE, PERSIST$\}$.
(2) **Privilege Escalation** (c_2): Acquiring access rights beyond the skill's declared capability set, exploiting misconfigured permissions or runtime privilege boundaries. $\Phi(c_2) = \{$CONFIGURE, EXECUTE$\}$.
(3) **Prompt Injection** (c_3): Adversarial content embedded in skill metadata, configuration templates, or tool return values that manipulates the agent's language model. $\Phi(c_3) = \{$LOAD, CONFIGURE, EXECUTE$\}$.
(4) **Dependency Confusion** (c_4): Publishing a public skill with the same name as a private internal skill, causing the resolver to fetch the malicious version. $\Phi(c_4) = \{$INSTALL$\}$.
(5) **Typosquatting** (c_5): Registering a skill name that is a near-homograph of a popular skill (e.g., `weahter-api` vs. `weather-api`). $\Phi(c_5) = \{$INSTALL$\}$.
(6) **Namespace Squatting** (c_6): Preemptively claiming a namespace likely to be used by a legitimate organization (e.g., `@google/search`). $\Phi(c_6) = \{$INSTALL$\}$.

The *total attack surface* is defined as the set of all (phase, class) pairs where an attack can manifest:

$$\mathcal{S}_{\text{attack}} = \big\{(\pi_j, c_i) \mid \pi_j \in \Phi(c_i)\big\}. \tag{1}$$

With our taxonomy, $|\mathcal{S}_{\text{attack}}| = 10$ distinct attack surfaces.

3.2.1 Threat Actors. We distinguish four categories of adversaries, ordered by increasing access and decreasing detectability:

Definition 3.3 (Threat Actor Categories)**.** A *threat actor* TA is characterized by its access vector and attack capability:

(1) **Malicious Author** (TA_1): Creates and publishes trojanized skills. Full control over skill content. Primary vector in ClawHavoc [68].
(2) **Compromised Registry** (TA_2): The attacker gains administrative control of a registry $r \in \mathcal{R}$ (e.g., through credential theft). Can modify any skill stored in r.
(3) **Supply Chain Attacker** (TA_3): Poisons a transitive dependency rather than the top-level skill. Analogous to the `event-stream` and `ua-parser` incidents in the npm ecosystem.
(4) **Insider Threat** (TA_4): An authorized user (developer or maintainer) who introduces malicious changes. Hardest to detect because access is legitimate.

Each threat actor can launch a subset of attack classes. TA_1 and TA_2 can launch all six classes. TA_3 is restricted to $\{c_1, c_2, c_3, c_4\}$ (supply chain attacks do not employ typosquatting or namespace squatting directly). TA_4 can launch $\{c_1, c_2, c_3\}$ (insider access bypasses install-time defenses).

3.3 The DY-Skill Attacker Model

We adapt the Dolev–Yao attacker model [19] to the agent skill supply chain. In the classical Dolev–Yao setting, an attacker controls the *network* and can intercept, inject, modify, and drop protocol messages. In DY-Skill, the "network" is the supply chain $\mathcal{SC}$, and "messages" are *skill packages*.

Definition 3.4 (Skill Message)**.** A *skill message* is a tuple $m = (name, ver, payload, caps)$ where:

- $name \in \Sigma^*$ is the skill's registered name,
- $ver \in \mathbb{V}$ is a semantic version identifier,
- $payload \in \{0,1\}^*$ is the skill's code and configuration, and
- $caps \subseteq \mathcal{CAP}$ is the set of declared capabilities (Section 4.1).

Definition 3.5 (DY-Skill Attacker)**.** A *DY-Skill attacker* Adv operates on a supply chain $\mathcal{SC}$ and maintains a *knowledge set* $K \subseteq \mathcal{M}$ (where $\mathcal{M}$ is the set of all skill messages) that is closed under six operations:

(1) **Intercept.** For any message m in transit through $\mathcal{SC}$: intercept(m) adds m to K and returns m. Formally, $K' = K \cup \{m\}$.

(2) **Inject.** For any message m constructible from K and any registry $r \in \mathcal{R}$: inject(m, r) adds m to r and to K. Formally, $r' = r \cup \{m\}$, $K' = K \cup \{m\}$.
(3) **Modify.** For any message m in transit through $\mathcal{SC}$ and any message m' constructible from K: modify(m, m') replaces m with $m' \neq m$. Formally, $\mathcal{SC}' = (\mathcal{SC} \setminus \{m\}) \cup \{m'\}$, $K' = K \cup \{m, m'\}$.
(4) **Drop.** For any message m in transit through $\mathcal{SC}$: drop(m) suppresses m. Formally, $\mathcal{SC}' = \mathcal{SC} \setminus \{m\}$, $K' = K \cup \{m\}$.
(5) **Forge Skills.** Adv can create skill definitions s' with arbitrary content, including manifests with arbitrary capability declarations $caps_{\text{declared}}$, executable code implementing capabilities $caps_{\text{actual}} \supseteq caps_{\text{declared}}$, and metadata mimicking legitimate skills (name squatting, version manipulation). $K' = K \cup \{s'\}$.
(6) **Compromise Registries.** Adv can modify skill entries in a registry $r \in \mathcal{R}$, subject to the constraint that Adv cannot forge valid cryptographic signatures under keys it does not possess. Formally, for $m \in r$, $r' = (r \setminus \{m\}) \cup \{m'\}$ where m' is constructible from K.

Operations (i)–(iv) are direct adaptations of the classical Dolev–Yao channel capabilities [19]. Operations (v) and (vi) are novel extensions specific to the agent skill supply chain, capturing the additional attack surface created by skill authoring and registry infrastructure.

The knowledge set K satisfies three invariants:

(a) *Monotonicity*: $K(t) \subseteq K(t+1)$ for all time steps t. Knowledge never decreases.
(b) *Interception closure*: For any observable message m in $\mathcal{SC}$, intercept(m) adds m to K.
(c) *Forge closure*: For any polynomial-time computable skill definition s', Adv can produce s' using operations (v) and (vi) and add it to K.

Perfect Sandbox Assumption. Analogous to the perfect cryptography assumption in classical Dolev–Yao, we assume that a correctly implemented capability sandbox cannot be broken without an *escape capability*. Formally: if a skill s is executed in a sandbox with capability set $Cap_D(s)$, then s cannot access any resource $r \notin Cap_D(s)$ unless s possesses a capability c such that $c.resource = r$. We relax this assumption in Section 5 when we discuss real-world sandbox implementations.

3.4 Theorem 1: DY-Skill Maximality

We now state and sketch the proof of our first main result: the DY-Skill attacker is maximally powerful in the symbolic model.

Theorem 3.6 (DY-Skill Maximality). *For any symbolic attacker $\mathcal{A}'$ operating on a supply chain $\mathcal{SC}$ under the perfect sandbox assumption, there exists a DY-Skill attacker* Adv *and a trace $\tau' = (\sigma_0, a_1, \sigma_1, \ldots, a_n, \sigma_n)$ such that for every observable event o in $\mathcal{A}'$'s execution, there is a corresponding event o' in τ' with effect(o) = effect(o').*

Proof sketch. We construct Adv by simulation, following the structure of Cervesato's maximality proof [9].

Base case. At $t = 0$, both $\mathcal{A}'$ and Adv have the same initial knowledge: the set of publicly available skills in $\mathcal{R}$. Adv intercepts all initial messages.

Inductive step. Suppose at time t, Adv's knowledge $K(t) \supseteq K'(t)$ (the knowledge of $\mathcal{A}'$). When $\mathcal{A}'$ performs an action a_{t+1}, we show Adv can produce the same observable effect:

- If a_{t+1} reads a message from $\mathcal{SC}$: Adv uses intercept.
- If a_{t+1} places a message into a registry: Adv uses inject.
- If a_{t+1} replaces a message in transit with a modified version: Adv uses modify.
- If a_{t+1} suppresses a message in transit: Adv uses drop.
- If a_{t+1} creates a new malicious skill definition: Adv uses forge-skills.
- If a_{t+1} modifies an existing registry entry: Adv uses compromise-registries.

By induction, the resulting trace τ' produces every observable effect of $\mathcal{A}'$. $K(t) \supseteq K'(t)$ is maintained by the monotonicity and closure properties of K.

The full proof, including the formal construction of the simulation relation and handling of composite operations, appears in Appendix A.1. □

Practical significance. Theorem 3.6 implies that any defense strategy that is secure against the DY-Skill attacker is secure against *all* symbolic attackers on the supply chain. This justifies using the DY-Skill model as the standard adversary for our static analysis (Section 4) and sandboxing (Section 5) frameworks. In particular, if SkillFortify's analysis proves that a skill is safe under the DY-Skill model, no weaker attacker can compromise it.

4 Static Analysis Framework

We present a static analysis framework grounded in abstract interpretation [14]. The framework operates in three phases: (1) capability inference via abstract interpretation over a capability lattice, (2) dangerous pattern detection using a catalog derived from real-world incidents, and (3) capability violation checking that compares inferred capabilities against declarations. The central guarantee is *soundness*: if the analysis reports no violations, the concrete execution cannot exceed declared capabilities.

4.1 Capability Lattice

We begin by defining the algebraic structure that underpins capability reasoning.

Definition 4.1 (Access Level Lattice). Let $\mathbb{L}_{cap} = (\{\text{NONE}, \text{READ}, \text{WRITE}, \text{ADMIN}\}, \sqsubseteq)$ be a totally ordered lattice (chain) with

$$\text{NONE} \sqsubseteq \text{READ} \sqsubseteq \text{WRITE} \sqsubseteq \text{ADMIN}.$$

The lattice operations are:

- *Join* (least upper bound): $a \sqcup b = \max(a, b)$.
- *Meet* (greatest lower bound): $a \sqcap b = \min(a, b)$.
- *Bottom*: $\bot = \text{NONE}$ (identity for $\sqcup$, absorbing for $\sqcap$).
- *Top*: $\top = \text{ADMIN}$ (identity for $\sqcap$, absorbing for $\sqcup$).

The four access levels model a standard escalation path. NONE denotes no access. READ permits observation but not modification. WRITE permits observation and mutation. ADMIN adds the authority to grant or revoke access for other principals, following the capability escalation model of Dennis and Van Horn [17].

Definition 4.2 (Resource Universe). The *resource universe* is the finite set

$C = \{\texttt{filesystem}, \texttt{network}, \texttt{environment}, \texttt{shell}, \texttt{skill\_invoke}, \texttt{clipboard}, \texttt{browser}, \texttt{database}\}$,

with $|C| = 8$. These resource types were identified by cross-referencing the empirical findings of Park et al. [54] (42,447 skills) and Wang et al. [63] (6,487 malicious tools).

Definition 4.3 (Capability and Capability Set). A *capability* is a pair $c = (r, \ell)$ where $r \in C$ is a resource type and $\ell \in \mathbb{L}_{cap}$ is an access level.

Capability $c_1 = (r_1, \ell_1)$ *subsumes* $c_2 = (r_2, \ell_2)$, written $c_2 \sqsubseteq c_1$, iff $r_1 = r_2$ and $\ell_2 \sqsubseteq \ell_1$.

A *capability set* $Cap(s)$ for a skill s is a function $Cap(s) : C \to \mathbb{L}_{cap}$ mapping each resource to an access level. Equivalently, $Cap(s)$ is an element of the product lattice $\mathbb{L}_{cap}^{|C|}$.

The product lattice ordering is pointwise: $Cap_1 \sqsubseteq Cap_2$ iff $Cap_1(r) \sqsubseteq Cap_2(r)$ for all $r \in C$. This structure is a complete lattice with $\bot = (\text{NONE}, \ldots, \text{NONE})$ and $\top = (\text{ADMIN}, \ldots, \text{ADMIN})$.

Lattice law verification. The lattice laws (idempotency, commutativity, associativity, absorption) hold by inspection since $\sqcup = \max$ and $\sqcap = \min$ on the totally ordered set $\{\text{NONE}, \text{READ}, \text{WRITE}, \text{ADMIN}\}$ satisfy these properties for any total order. Product lattice completeness follows from Birkhoff [5]: the product of complete lattices is complete.

4.2 Abstract Skill Semantics

We define the concrete and abstract semantics of a skill and establish a Galois connection between them.

Definition 4.4 (Concrete Skill Semantics). Let Σ be the set of concrete program states (memory contents, file handles, network sockets, environment bindings). The *concrete semantics* of a skill s is a function $[\![s]\!] : \wp(\Sigma) \to \wp(\Sigma)$ that maps a set of initial states to the set of reachable final states.

Definition 4.5 (Abstract Skill Semantics). The *abstract semantics* of s is a function $[\![s]\!]^\sharp : \mathbb{L}_{cap}^{|C|} \to \mathbb{L}_{cap}^{|C|}$ that over-approximates the concrete semantics in the capability domain.

We relate the two via a Galois connection, following Cousot and Cousot [14].

Definition 4.6 (Galois Connection). Let $(\wp(\Sigma), \subseteq)$ be the concrete domain and $(\mathbb{L}_{cap}^{|C|}, \sqsubseteq)$ be the abstract domain. The pair of functions (α, γ) forms a Galois connection:

$$\alpha : \wp(\Sigma) \to \mathbb{L}_{cap}^{|C|}, \qquad \gamma : \mathbb{L}_{cap}^{|C|} \to \wp(\Sigma),$$

satisfying for all $S \in \wp(\Sigma)$ and all $a \in \mathbb{L}_{cap}^{|C|}$:

$$\alpha(S) \sqsubseteq a \iff S \subseteq \gamma(a). \tag{2}$$

The abstraction function α extracts the capability footprint: for each resource $r \in C$, $\alpha(S)(r)$ is the least $\ell \in \mathbb{L}_{cap}$ such that every state in S accesses r at level ℓ or below. Concretization $\gamma(a)$ returns all states whose resource accesses are bounded by a.

Proposition 4.7 (Soundness of Abstraction). *If $\alpha \circ [\![s]\!] \sqsubseteq [\![s]\!]^\sharp \circ \alpha$, then the abstract fixpoint over-approximates the concrete fixpoint:*

$$\alpha(\text{lfp } [\![s]\!]) \sqsubseteq \text{lfp } [\![s]\!]^\sharp. \tag{3}$$

This is a direct instance of the fundamental theorem of abstract interpretation [14]. Concretely, it means: if the abstract analysis concludes that s accesses resource r at level ℓ, then every concrete execution of s accesses r at level ℓ or below. False positives (the abstract analysis may overestimate access) are possible; false negatives (the analysis misses an access) are not.

4.3 Three-Phase Analysis

The SkillFortify static analyzer operates in three sequential phases.

Phase 1: Capability Inference (Abstract Interpretation). Given a parsed skill s with content fields (instructions, shell commands, URLs, environment references, code blocks), the analyzer computes the inferred capability set $Cap_I(s) \in \mathbb{L}_{cap}^{|C|}$ by applying $[\![s]\!]^\sharp$.

Concretely, the inference is a conservative over-approximation:

- URL references $\Rightarrow$ `network` $\geq$ READ; HTTP write methods (POST, PUT, PATCH, DELETE) $\Rightarrow$ `network` $\geq$ WRITE.
- Shell command invocations $\Rightarrow$ `shell` $\geq$ WRITE.
- Environment variable references $\Rightarrow$ `environment` $\geq$ READ.
- File-operation keywords $\Rightarrow$ `filesystem` $\geq$ READ or WRITE depending on the operation.

Each rule is a *transfer function* in the abstract domain: it maps pattern presence to a lower bound on the capability requirement.

Lemma 4.8 (Transfer Function Soundness). *Each transfer function in Phase 1 satisfies the Galois connection condition: for every set of concrete states S matching the detected pattern, $\alpha(S) \sqsubseteq \tau^\sharp(\alpha(S))$, where $\tau^\sharp$ is the abstract transfer function. Table 2 verifies this for each resource type.*

Table 2: Galois connection verification for each transfer function. For each pattern type, we show the inferred capability (abstract), the set of concrete operations it covers, and the containment relationship $\alpha(\text{concrete}) \sqsubseteq \text{abstract}$.

Pattern	Abstract $\tau^\sharp$	Concrete $\gamma(\tau^\sharp)$	Sound
URL reference	(net, READ)	States with HTTP GET access	✓
HTTP write verb	(net, WRITE)	States with HTTP POST/PUT/…	✓
Shell command	(shell, WRITE)	States executing shell	✓
Env var reference	(env, READ)	States reading env vars	✓
File read keyword	(fs, READ)	States reading files	✓
File write keyword	(fs, WRITE)	States writing files	✓
Sub-skill invoke	(skill, WRITE)	States invoking skills	✓
DB access keyword	(db, READ/WRITE)	States accessing database	✓

In each row, the concretization of the abstract result ($\gamma(\tau^\sharp)$) is a superset of the set of concrete states matching the pattern. This is because the abstract transfer function assigns the *maximum* access level that any concrete operation in the category could require, which is the defining property of a sound over-approximation under the Galois connection (Definition 4.6).

Phase 2: Dangerous Pattern Detection. The analyzer matches skill content against a catalog of $|\mathcal{P}|$ threat patterns derived from ClawHavoc, MalTool, and "Agent Skills in the Wild." Each pattern $p_j \in \mathcal{P}$ is a triple (r_j, σ_j, c_j) where r_j is a compiled regular expression, $\sigma_j \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}, \text{CRITICAL}\}$ is a severity, and c_j is an attack class from Definition 3.2.

Pattern examples include:

- `curl` piped to shell ($\sigma = \text{CRITICAL}, c = c_2$),
- Base64 decode piped to shell ($\sigma = \text{CRITICAL}, c = c_2$),
- Netcat listener ($\sigma = \text{CRITICAL}, c = c_1$),
- Dynamic code evaluation ($\sigma = \text{HIGH}, c = c_2$).

Additionally, a cross-channel *information flow* check detects the combination of base64 encoding and external network access—a pattern empirically associated with data exfiltration [63].

Phase 3: Capability Violation Check. If the skill declares a capability set $Cap_D(s)$, the analyzer checks:

$$Cap_I(s) \sqsubseteq Cap_D(s). \tag{4}$$

Any resource r where $Cap_I(s)(r) \not\sqsubseteq Cap_D(s)(r)$ constitutes a *capability violation*—the skill requires more authority than it claims.

The set of violations is:

$$Viol(s) = \{r \in C \mid Cap_I(s)(r) \not\sqsubseteq Cap_D(s)(r)\}. \tag{5}$$

4.4 Theorem 2: Analysis Soundness

The following theorem is the central guarantee of SkillFortify's static analysis: it establishes that the analysis is *sound*—if it certifies a skill as compliant, the skill truly cannot exceed its declared capabilities.

Theorem 4.9 (Analysis Soundness). *Let s be a skill with declared capability set $Cap_D(s)$. If the abstract analysis reports no violations—i.e., $Viol(s) = \emptyset$—then for every concrete execution trace τ of s and every resource access (r, ℓ) performed in τ:*

$$\ell \sqsubseteq Cap_D(s)(r). \tag{6}$$

That is, the concrete execution never accesses any resource beyond the declared capability level.

Proof sketch. The argument proceeds in two steps.

Step 1: Abstraction soundness. By the Galois connection (Definition 4.6) and Proposition 4.7, the inferred capability set $Cap_I(s)$ over-approximates the actual resource accesses of s:

$$\forall\, \tau,\ \forall\, (r, \ell) \in \tau\colon \ell \sqsubseteq Cap_I(s)(r).$$

Step 2: Violation check. If $Viol(s) = \emptyset$, then $Cap_I(s) \sqsubseteq Cap_D(s)$, i.e., $Cap_I(s)(r) \sqsubseteq Cap_D(s)(r)$ for all $r \in C$.

Combining. By transitivity of $\sqsubseteq$ on $\mathbb{L}_{cap}$:

$$\ell \sqsubseteq Cap_I(s)(r) \sqsubseteq Cap_D(s)(r) \qquad \text{for all } (r, \ell) \in \tau.$$

The full proof, including the construction of the Galois connection and the fixpoint argument, appears in Appendix A.2. □

Practical significance. Theorem 4.9 distinguishes SkillFortify from all existing heuristic scanners. Cisco's `skill-scanner` [12] explicitly warns that "no findings does not mean no risk." Our guarantee is the opposite: *if SkillFortify reports no violations for the analyzed properties, there are no violations in any concrete execution.* This is the same class of guarantee that the Astrée analyzer [15] provides for Airbus A380 flight software—sound over-approximation with zero false negatives.

Limitations. The analysis is sound but not complete: false positives (flagging a safe skill as potentially dangerous) are possible because the abstract domain over-approximates. Additionally, the current analysis covers statically observable patterns; skills that dynamically construct network URLs or shell commands at runtime may require dynamic analysis (Section 11).

5 Capability-Based Sandboxing

The static analysis of Section 4 verifies skill behavior *before* execution. In this section, we complement static verification with a runtime enforcement mechanism based on the capability model of Dennis and Van Horn [17] and its object-capability (ocap) extension by Miller [44]. The central guarantee is *capability confinement*: a sandboxed skill cannot access any resource beyond its declared capability set, even if the skill attempts to escalate privileges at runtime.

5.1 Capability Model for Agent Skills

We ground the sandbox in the object-capability model, where capabilities are unforgeable references that couple resource designations with access rights.

Definition 5.1 (Skill Capability)**.** A *skill capability* is a pair $c = (r, \ell)$ where $r \in C$ (Definition 4.2) is a resource type and $\ell \in \mathbb{L}_{cap}$ (Definition 4.1) is an access level. A capability is *unforgeable*: a skill can only obtain a capability through the four mechanisms of Dennis and Van Horn [17]:

(1) **Initial conditions**—held at sandbox creation,
(2) **Parenthood**—the creator obtains the only reference,
(3) **Endowment**—the creator grants a subset of its own capabilities, and
(4) **Introduction**—a capability holder introduces two other principals via a shared reference.

Capabilities in this model are the *only* source of authority. There is no ambient authority—a skill cannot access a resource simply because it "exists" in the environment. This is the defining property of a *capability-safe* system [41].

Definition 5.2 (Declared Capability Set)**.** For a skill s, the *declared capability set* $Cap_D(s) \subseteq C \times \mathbb{L}_{cap}$ is the set of capabilities that s declares in its manifest. This set is specified by the skill author and verified by SkillFortify's static analysis (Section 4.3) before the skill is installed.

Definition 5.3 (Required Capability)**.** For an action a performed by a skill during execution, the *required capability* $req(a) = (r, \ell)$ is the minimum capability needed to perform a. For example, reading a file requires (`filesystem`, READ); executing a shell command requires (`shell`, WRITE).

The sandbox mediates every action a by checking:

$$\exists\, (r', \ell') \in Cap_D(s)\colon r' = req(a).r \ \wedge\ req(a).\ell \sqsubseteq \ell'. \tag{7}$$

If Equation (7) fails, the action is denied and a security finding is raised.

5.2 Capability Attenuation

Agent skills frequently delegate work to sub-skills (e.g., a "research assistant" skill invokes a "web search" skill and a "file writer" skill). The capability model must ensure that delegation does not amplify authority.

Definition 5.4 (Capability Attenuation)**.** When a parent skill s_p delegates to a child skill s_c:

$$Cap_D(s_c) \subseteq Cap_D(s_p). \tag{8}$$

That is, the child can receive at most the capabilities the parent already possesses. This implements the *Principle of Least Authority* (POLA) [44]: each skill should receive exactly the capabilities it needs and no more.

Formally, for each resource $r \in C$:

$$Cap_D(s_c)(r) \sqsubseteq Cap_D(s_p)(r).$$

This is the *no authority amplification* property of capability systems [45]: no skill can grant a capability it does not possess. Combined with unforgeability, this bounds the transitive authority of any delegation chain.

Proposition 5.5 (Transitive Attenuation). *For a delegation chain $s_1 \to s_2 \to \cdots \to s_k$:*

$$Cap_D(s_k) \subseteq Cap_D(s_{k-1}) \subseteq \cdots \subseteq Cap_D(s_1).$$

Authority can only decrease along a delegation chain; it can never increase.

Proof. Immediate by induction on the chain length using Equation (8). □

5.3 CapabilitySet Verification Operations

The sandbox exposes three verification operations that implement the formal model.

Operation 1: Permits(c_{req}). Given a required capability $c_{req} = (r, \ell)$, check whether the declared set covers it:

$$\textsc{Permits}(Cap_D, c_{req}) = \begin{cases} true & \text{if } \exists\, (r', \ell') \in Cap_D : r' = r \wedge \ell \sqsubseteq \ell', \\ false & \text{otherwise.} \end{cases}$$

Operation 2: IsSubsetOf(Cap_1, Cap_2). Formal POLA verification—check that one capability set is dominated by another:

$$\textsc{IsSubsetOf}(Cap_1, Cap_2) = \forall\, (r, \ell) \in Cap_1 : \textsc{Permits}(Cap_2, (r, \ell)).$$

This is used both at install-time (checking that inferred capabilities do not exceed declared, $Cap_I \sqsubseteq Cap_D$) and at delegation-time (checking $Cap_D(s_c) \sqsubseteq Cap_D(s_p)$).

Operation 3: Violations(Cap_I, Cap_D). Return the set of capabilities that exceed declared bounds:

$$\textsc{Violations}(Cap_I, Cap_D) = \{(r, \ell) \in Cap_I \mid \neg\, \textsc{Permits}(Cap_D, (r, \ell))\}.$$

An empty violation set constitutes a formal proof that the skill respects its declared capability boundary: $\textsc{Violations} = \emptyset \implies Cap_I \sqsubseteq Cap_D$.

5.4 Theorem 3: Capability Confinement

We state the capability confinement guarantee in two parts. Theorem 3a provides the *static* guarantee that SkillFortify v1 delivers: if the static analysis finds no violations, then no operation in the skill's code exceeds declared capabilities in the abstract domain. Theorem 3b provides the *runtime design* guarantee for future sandbox implementations.

Theorem 5.6 (Static Capability Confinement). *Let s be a skill with declared capability set $Cap_D(s)$. If the static analysis reports no violations—i.e., $Viol(s) = \emptyset$—then for every code construct in the skill's definition, no operation o in s requires a capability exceeding the declared level. Formally:*

$$\forall\, o \in ops(s) : cap(o) \sqsubseteq \bigsqcup Cap_D(s). \tag{9}$$

Proof sketch. We proceed by structural induction on the syntax of the skill's code. For each code construct—assignment, conditional, loop, function call—we show that the capability requirements of all reachable operations are bounded by the inferred capability set, which in turn is bounded by the declared set when $Viol(s) = \emptyset$.

Base case (single operation o): The operation o has capability requirement $cap(o)$. The abstract analysis infers $cap(o) \sqsubseteq Cap_I(s)(r)$ for the relevant resource r (by soundness of the transfer functions, Theorem 4.9). Since $Viol(s) = \emptyset$, we have $Cap_I(s) \sqsubseteq Cap_D(s)$, so $cap(o) \sqsubseteq Cap_D(s)(r)$.

Sequential composition (s_1; s_2): By the induction hypothesis, all operations in s_1 and s_2 individually satisfy the bound. Their union does not introduce new operations, so the bound holds for the composition.

Conditional (`if` b `then` s_1 `else` s_2): The inferred capabilities include both branches. Any operation reachable at runtime belongs to one branch and is bounded by the induction hypothesis.

Loop (`while` b `do` s_{body}): The inferred capabilities of the loop equal those of the body (repetition does not introduce new capabilities). By the induction hypothesis on the body, the bound holds.

The full proof with all cases appears in Appendix A.3. □

Theorem 5.7 (Runtime Capability Confinement—Design). *If a runtime sandbox implementing the capability-safe execution model (no ambient authority, unforgeable capabilities, capability check on every action per Equation (7)) is deployed, then for any skill s with declared capability set $Cap_D(s)$ and for any action a performed by s during execution:*

$$req(a).\ell \sqsubseteq Cap_D(s)(req(a).r). \tag{10}$$

Theorem 5.7 is a *design theorem*: it specifies the guarantee that a correctly implemented runtime sandbox must provide. The proof relies on the capability-safe execution model of Dennis and Van Horn [17] and the object-capability confinement result of Maffeis, Mitchell, and Taly [41]. We include a proof sketch in Appendix A.3 and note that SkillFortify v1 provides the *static* guarantee (Theorem 5.6); runtime enforcement is a design contribution for future work.

Relation to Maffeis, Mitchell & Taly (2010). Theorem 5.7 instantiates the *capability safety theorem* of Maffeis et al. [41] in the agent skill domain. Their result establishes that in a capability-safe language (JavaScript strict mode in their formalization), untrusted code is confined to the authority explicitly passed to it. We extend this to agent skills, where the "language" is the skill execution runtime and the "authority" is the declared capability set.

Combined guarantee. Theorems 4.9 and 5.6 provide the defense currently implemented in SkillFortify:

- *Analysis soundness* (Theorem 4.9): The inferred capabilities over-approximate the concrete resource accesses.

- *Static confinement* (Theorem 5.6): If no violations are found, all operations in the skill's code are within declared capabilities.

Together with the runtime design theorem (Theorem 5.7), these provide defense in depth: static verification before installation, with the option of runtime enforcement during execution. A skill that passes the static check is provably confined in the abstract domain: it cannot exfiltrate data (c_1), escalate privileges (c_2), or abuse undeclared resources, under the DY-Skill attacker model (Section 3.3).

Scope of guarantees. SkillFortify v1 provides static analysis guarantees (Theorems 4.9 and 5.6). The runtime confinement guarantee (Theorem 5.7) specifies the contract for a future sandbox implementation. This separation is intentional: static analysis catches capability violations before installation (the primary defense), while runtime sandboxing provides a safety net for dynamically constructed operations that static analysis cannot fully resolve (see Section 11).

6 Agent Dependency Graph and Lockfile Semantics

Sections 3–5 established how to *detect* and *confine* individual malicious skills. In practice, agent configurations rarely consist of a single skill in isolation: modern agents compose dozens of skills, each of which may itself depend on other skills, shared libraries, or external MCP servers. A single compromised transitive dependency can undermine the security of the entire installation, as demonstrated by the SolarWinds [11] and Log4Shell [3] incidents in traditional software supply chains, and by the ClawHavoc campaign [68] in the agent ecosystem.

This section introduces the *Agent Dependency Graph* (ADG), a formal data structure that captures skills, their version spaces, inter-skill dependencies, conflicts, and per-version capability requirements. We present a SAT-based resolution algorithm that simultaneously satisfies dependency, conflict, and security constraints, and we define *lockfile semantics* that guarantee reproducible, tamper-detectable installations. We conclude with an Agent Skill Bill of Materials (ASBOM) output aligned with the CycloneDX 1.6 standard [53].

6.1 Agent Dependency Graph (ADG)

We model the agent skill ecosystem as a *package universe* extended with security metadata, following the formalism of Mancinelli et al. [42] and Tucker et al. [61].

Definition 6.1 (Agent Dependency Graph)**.** An *Agent Dependency Graph* is a tuple $ADG = (\mathcal{S}, V, D, C, Cap)$ where:

- $\mathcal{S}$ is a finite set of *skill names*.
- $V : \mathcal{S} \to 2^{\mathbb{N}}$ maps each skill to its set of available versions, ordered by semantic versioning precedence [56].
- $D : \mathcal{S} \times \mathbb{N} \to 2^{\mathcal{S} \times Constraint}$ maps each skill-version pair (s, v) to a set of *dependency edges*, where each edge (q, γ) requires that some version of skill q satisfying constraint γ be co-installed.
- $C : \mathcal{S} \times \mathbb{N} \to 2^{\mathcal{S} \times Constraint}$ maps each skill-version pair to a set of *conflict edges*: if $(q, \gamma) \in C(s, v)$, then no version of q satisfying γ may be co-installed with (s, v).
- $Cap : \mathcal{S} \times \mathbb{N} \to 2^{C}$ maps each skill-version pair to the set of capabilities it requires at runtime, where C is the capability universe from Definition 4.3.

The ADG extends the classical package universe of Mancinelli et al. [42] with the capability map Cap. This extension is critical: it enables the resolver to reject skill-versions that demand capabilities exceeding the security policy, *before* installation occurs. Traditional package managers lack this dimension entirely.

6.2 Version Constraints

Definition 6.2 (Version Constraint)**.** A *version constraint* γ is a predicate over version numbers. We support the following atomic forms, mirroring PEP 440 and SemVer conventions:

$$\gamma ::= == v \mid != v \mid >= v \mid <= v \mid > v \mid < v \mid \hat{} v \mid \tilde{} v \mid *$$

Compound constraints are formed by conjunction: $\gamma_1 \wedge \gamma_2 \wedge \cdots \wedge \gamma_k$. A version w *satisfies* a compound constraint iff it satisfies every atomic constituent.

The *caret* operator $\hat{} v$ permits any version with the same major number and $\geq v$ (or, if the major number is 0, the same major and minor). The *tilde* operator $\tilde{} v$ permits any version with the same major and minor number and patch $\geq$ the target patch. The *wildcard* $*$ is trivially satisfied by any version. These semantics follow standard package manager conventions [24].

6.3 SAT-Based Dependency Resolution

Dependency resolution for package managers is NP-complete, reducible from 3-SAT [42]. We follow the OPIUM approach [61] and encode the resolution problem as a propositional satisfiability instance, solved by a modern Conflict-Driven Clause Learning (CDCL) solver.

Definition 6.3 (SAT Encoding)**.** Given an ADG $(\mathcal{S}, V, D, C, Cap)$ and a set of *allowed capabilities* $\mathcal{A} \subseteq C$, we introduce a Boolean variable $x_{s,v}$ for each skill $s \in \mathcal{S}$ and version $v \in V(s)$. Variable $x_{s,v} = true$ means "skill s at version v is included in the installation." The following clauses encode the constraints:

(C1) At-most-one version per skill. For each skill s and every pair of distinct versions $v_i, v_j \in V(s)$:

$$\neg x_{s,v_i} \vee \neg x_{s,v_j} \tag{11}$$

(C2) Root requirements. For each user-specified requirement (s, γ):

$$\bigvee_{\{w \in V(s) \mid w \models \gamma\}} x_{s,w} \tag{12}$$

(C3) Dependency implications. For each (s, v) and each dependency $(q, \gamma) \in D(s, v)$:

$$\neg x_{s,v} \vee \bigvee_{\{w \in V(q) \mid w \models \gamma\}} x_{q,w} \tag{13}$$

If no version of q satisfies γ, this degenerates to a unit clause $\neg x_{s,v}$, effectively banning (s, v).

(C4) Conflict exclusions. For each (s, v) and each conflict $(q, \gamma) \in C(s, v)$, and for each $w \in V(q)$ with $w \models \gamma$:

$$\neg x_{s,v} \vee \neg x_{q,w} \tag{14}$$

(C5) Capability bounds. For each (s, v) such that $Cap(s, v) \not\subseteq \mathcal{A}$:

$$\neg x_{s,v} \tag{15}$$

The key novelty over classical package resolution is clause family (C5): it integrates the static capability analysis of Section 4 directly into the resolution process, ensuring that no installation plan can include a skill-version whose runtime requirements exceed the declared security policy.

Implementation. We use the Glucose3 CDCL solver via the PySAT framework [33]. At-most-one constraints are encoded as pairwise negation (Eq. 11), which is compact for the typical case of $|V(s)| \leq 20$ versions per skill. For larger version spaces, sequential counter encodings from the PySAT cardinality module can be substituted.

6.4 Secure Installation

Definition 6.4 (Secure Installation)**.** An *installation* $I \subseteq \mathcal{S} \times \mathbb{N}$ is a set of skill-version pairs. I is *secure* with respect to $(\mathcal{S}, V, D, C, Cap, \mathcal{A})$ if and only if:

(1) **Completeness.** For every $(s, v) \in I$ and every $(q, \gamma) \in D(s, v)$, there exists $(q, w) \in I$ such that $w \models \gamma$.
(2) **Consistency.** For every $(s, v) \in I$ and every $(q, \gamma) \in C(s, v)$, there is no $(q, w) \in I$ with $w \models \gamma$.
(3) **Capability safety.** For every $(s, v) \in I$: $Cap(s, v) \subseteq \mathcal{A}$.

Condition (3) is the agent-specific extension: it ties the installation plan to the capability-based sandboxing model of Section 5, ensuring that every installed skill respects the declared security boundary. In traditional package managers (apt, pip, npm), no analogue of this condition exists.

6.5 Lockfile Semantics

A *lockfile* is the deterministic serialization of a secure installation. We define the `skill-lock.json` format to provide three guarantees: reproducibility, integrity, and auditability.

Definition 6.5 (Lockfile)**.** A *lockfile* $\mathcal{L}$ is a tuple $\mathcal{L} = (I, H, M)$ where:

- $I \subseteq \mathcal{S} \times \mathbb{N}$ is a secure installation (Definition 6.4).
- $H : I \to \{0, 1\}^{256}$ maps each installed skill-version to its SHA-256 content hash.
- M is a metadata record containing the resolution strategy, the allowed capability set $\mathcal{A}$, a generation timestamp, and the tool version.

Definition 6.6 (Lockfile Reproducibility)**.** A resolver $\mathcal{R}$ is *reproducible* if, for any ADG and requirement set, running $\mathcal{R}$ twice on the same input produces identical lockfiles:

$$\mathcal{R}(ADG, reqs, \mathcal{A}) = \mathcal{R}(ADG, reqs, \mathcal{A})$$

We achieve reproducibility by fixing the variable ordering in the SAT encoding (lexicographic by skill name, then descending version) and using a deterministic solver configuration. The lockfile itself is serialized with sorted keys and canonical JSON formatting, ensuring byte-identical output for identical inputs.

Definition 6.7 (Integrity Verification)**.** At installation time, for each $(s, v) \in I$, the installer computes the SHA-256 hash of the skill content and verifies:

$$\texttt{sha256}(content(s, v)) = H(s, v)$$

A mismatch indicates tampering: either the skill source was modified after resolution (supply chain attack), or the lockfile was corrupted.

This mechanism is analogous to the integrity fields in `package-lock.json` (npm) and `Cargo.lock` (Rust), adapted for the agent skill ecosystem where no prior lockfile standard exists.

6.6 Agent Skill Bill of Materials (ASBOM)

Regulatory frameworks including the EU AI Act (Article 17, technical documentation requirements) [22] and the NIST AI Risk Management Framework [47] increasingly require transparency about third-party AI components. We generate an *Agent Skill Bill of Materials* (ASBOM) as a structured inventory of all skills in an agent's dependency closure.

Definition 6.8 (Agent Skill Bill of Materials)**.** An ASBOM is a CycloneDX 1.6-compatible [53] document that records, for each $(s, v) \in I$:

- **Identity:** skill name, version, format (Claude, MCP, OpenClaw), and content hash.
- **Capabilities:** the set $Cap(s, v)$ of declared runtime permissions.
- **Dependencies:** resolved dependency edges $\{(q, w) \mid (q, \gamma) \in D(s, v),\ (q, w) \in I,\ w \models \gamma\}$.
- **Trust metadata:** trust score, trust level (Section 7), and provenance information.
- **Vulnerability status:** known CVEs or analysis findings affecting (s, v).

The CycloneDX format enables integration with existing enterprise vulnerability management platforms (e.g., Dependency-Track, Grype) and provides the machine-readable evidence required for compliance audits.

6.7 Transitive Vulnerability Propagation

A critical advantage of the graph-based model is the ability to propagate vulnerability information transitively.

Definition 6.9 (Affected Installation)**.** Given a set of known-vulnerable skill-versions $Vuln \subseteq \mathcal{S} \times \mathbb{N}$, a skill-version $(s, v) \in I \setminus Vuln$ is *transitively affected* if there exists a path in the dependency graph from (s, v) to some $(q, w) \in Vuln$. The set of affected installations is:

$$Affected(I, Vuln) = \{(s, v) \in I \setminus Vuln \mid \exists\, (q, w) \in Vuln\ .\ (s, v) \rightsquigarrow^* (q, w)\}$$

where $\rightsquigarrow^*$ denotes transitive reachability through dependency edges.

We compute *Affected* by BFS over the reverse dependency graph (from vulnerable nodes to their dependents), analogous to how `npm audit` identifies transitively affected packages. The difference is that our propagation also considers *capability implications*: if a vulnerable skill has capability $c \in Cap(q, w)$, then all skills in its reverse transitive closure that delegate or depend on capability c are flagged with elevated severity.

6.8 Theorem 4: Resolution Soundness

Theorem 6.10 (Resolution Soundness). *Let $ADG = (\mathcal{S}, V, D, C, Cap)$ be an Agent Dependency Graph and $\mathcal{A} \subseteq C$ a set of allowed capabilities. Let Φ be the conjunction of all clauses (C1)–(C5) from Definition 6.3. Then:*

(1) Φ is satisfiable if and only if a secure installation (Definition 6.4) exists.

(2) Any satisfying assignment $\sigma \models \Phi$ induces a secure installation $I_\sigma = \{(s, v) \mid \sigma(x_{s,v}) = true\}$, and the corresponding lockfile $\mathcal{L}_\sigma = (I_\sigma, H, M)$ satisfies all dependency, conflict, and capability constraints.

Proof sketch. We establish both directions.

($\Rightarrow$) Suppose $\sigma \models \Phi$. Define $I_\sigma = \{(s, v) \mid \sigma(x_{s,v}) = true\}$.

- *At-most-one:* By clause family (C1), for each skill s at most one variable $x_{s,v}$ is true, so I_σ contains at most one version per skill.
- *Completeness:* Fix $(s, v) \in I_\sigma$ and $(q, \gamma) \in D(s, v)$. Since $\sigma(x_{s,v}) = true$, clause (C3) requires $\sigma(x_{q,w}) = true$ for some $w \models \gamma$. Hence $(q, w) \in I_\sigma$.
- *Consistency:* Fix $(s, v) \in I_\sigma$ and $(q, \gamma) \in C(s, v)$. For every $w \models \gamma$, clause (C4) gives $\neg x_{s,v} \vee \neg x_{q,w}$. Since $\sigma(x_{s,v}) = true$, we have $\sigma(x_{q,w}) = false$, so $(q, w) \notin I_\sigma$.
- *Capability safety:* If $Cap(s, v) \not\subseteq \mathcal{A}$, clause (C5) forces $\sigma(x_{s,v}) = false$, so $(s, v) \notin I_\sigma$. Contrapositive: $(s, v) \in I_\sigma \Rightarrow Cap(s, v) \subseteq \mathcal{A}$.

($\Leftarrow$) Suppose I is a secure installation. Define $\sigma(x_{s,v}) = [(s, v) \in I]$ (Iverson bracket).

- *(C1):* At most one version per skill in I (by definition of installation), so pairwise exclusion holds.
- *(C2):* For each requirement (s, γ), completeness of I implies some $(s, w) \in I$ with $w \models \gamma$, so the disjunction is satisfied.
- *(C3):* For each $(s, v) \in I$ and $(q, \gamma) \in D(s, v)$, completeness gives $(q, w) \in I$ with $w \models \gamma$, making the implication clause true. If $(s, v) \notin I$, the clause is trivially satisfied via $\neg x_{s,v}$.
- *(C4):* Consistency of I ensures no conflicting pair is co-installed; the mutual exclusion clauses hold.
- *(C5):* Capability safety of I ensures $Cap(s, v) \subseteq \mathcal{A}$ for all $(s, v) \in I$, so no unit clause $\neg x_{s,v}$ is violated.

Thus $\sigma \models \Phi$, and Φ is satisfiable. □

Remark 6.11. The bidirectional correspondence in Theorem 6.10 means that the SAT solver provides a *complete* decision procedure for secure installability: if the solver reports UNSAT, then *no* secure installation exists under the given constraints. In such cases, the conflict analysis phase of the CDCL solver yields an unsatisfiable core, which we translate into human-readable diagnostic messages explaining which dependency, conflict, or capability constraint caused the failure.

Complexity. The dependency resolution problem is NP-complete in general [42]. Our capability-bounded extension (clause family C5) adds only $O(|\mathcal{S}| \cdot \max_s |V(s)|)$ unit clauses, so the theoretical worst case is unchanged. In practice, the agent skill ecosystem has $|\mathcal{S}| \leq 10^3$ and $|V(s)| \leq 20$, well within the efficient regime of modern CDCL solvers: our experiments (Section 9) show resolution times under 0.5 seconds for 500 skills.

Table 3: Agent Dependency Graph vs. traditional package dependency resolution.

Feature	Traditional	ADG (Ours)
Version constraints	✓	✓
Conflict detection	✓	✓
SAT-based resolution	✓	✓
Capability bounding	–	✓
Trust-aware resolution	–	✓
Lockfile integrity	✓	✓
ASBOM generation	Partial	✓
Vulnerability propagation	✓	✓
Capability-aware propagation	–	✓

Comparison with traditional package managers. Table 3 summarizes the differences between our ADG-based resolver and traditional package dependency resolution.

7 Trust Score Algebra

The preceding sections established mechanisms for verifying individual skills (Sections 4–5) and resolving their dependencies securely (Section 6). A complementary question remains: *how much should a developer trust a given skill?* Binary safe/unsafe classification is insufficient in practice—skills exist on a spectrum from unsigned, anonymous submissions to formally verified, organization-signed artifacts with years of community track record.

This section introduces a *trust score algebra* that combines heterogeneous trust signals into a single composite score, propagates trust conservatively through dependency chains, models trust decay for unmaintained skills, and maps numeric scores to graduated assurance levels inspired by the SLSA framework [26]. Our construction draws on the trust management literature, particularly the PolicyMaker and KeyNote systems [6, 7], and the formal trust models of Carbone et al. [8] and Jøsang [35].

7.1 Trust Score Model

Definition 7.1 (Trust Signals)**.** For a skill s at version v, we define four orthogonal *trust signals*, each taking values in the closed interval $[0, 1]$:

(1) $T_p(s, v) \in [0, 1]$: **Provenance.** Measures author verification and signing status. $T_p = 0$ for unsigned, anonymous authors; $T_p = 1$ for organization-signed skills with reproducible builds and verified identity.

(2) $T_b(s, v) \in [0, 1]$: **Behavioral.** Measures the outcome of static analysis (Section 4). $T_b = 0$ if critical capability violations are detected; $T_b = 1$ if analysis reports no findings.

(3) $T_c(s, v) \in [0, 1]$: **Community.** Aggregates community trust signals: usage count, age, issue reports, and peer reviews. $T_c = 0$ for a brand-new skill with no usage; $T_c = 1$ for a widely adopted, mature skill with no reported issues.

(4) $T_h(s, v) \in [0, 1]$: **Historical.** Captures the skill's past vulnerability record. $T_h = 0$ if the skill has a history of CVEs or security incidents; $T_h = 1$ for a clean historical record.

We write $\mathbf{T}(s, v) = (T_p, T_b, T_c, T_h) \in [0, 1]^4$ for the signal vector.

The four signals are designed to be *orthogonal*: provenance measures *who* published the skill, behavioral measures *what* the skill does, community measures *how widely* it is adopted, and historical measures *how reliable* it has been over time.

Definition 7.2 (Trust Weights)**.** A *weight vector* $\mathbf{w} = (w_p, w_b, w_c, w_h) \in \mathbb{R}^4_{\geq 0}$ satisfies:

$$w_p + w_b + w_c + w_h = 1 \qquad \text{and} \qquad w_p, w_b, w_c, w_h \geq 0$$

The default weight vector is $\mathbf{w}_0 = (0.3, 0.3, 0.2, 0.2)$, reflecting equal priority for provenance and behavioral analysis (the signals most directly under the tool's and author's control), with lower weight for community and historical signals (which accumulate over time).

Definition 7.3 (Intrinsic Trust Score)**.** The *intrinsic trust score* of skill s at version v is the weighted linear combination:

$$T(s, v) = w_p \cdot T_p(s, v) + w_b \cdot T_b(s, v) + w_c \cdot T_c(s, v) + w_h \cdot T_h(s, v) \quad (16)$$

Since all weights are non-negative and sum to 1, and all signals are in $[0, 1]$, the score satisfies $T(s, v) \in [0, 1]$.

7.2 Trust Propagation

A skill's effective trustworthiness depends not only on its own properties but on the trustworthiness of its dependencies. A formally verified skill that depends on an unsigned, unvetted library inherits risk from that dependency. We model this through *multiplicative trust propagation*, following the transitive trust composition approach of Carbone et al. [8].

Definition 7.4 (Trust Composition Operator)**.** The *trust composition operator* $\otimes\colon [0, 1] \times [0, 1] \to [0, 1]$ is defined as:

$$a \otimes b = a \cdot b \quad (17)$$

This is conservative: composed trust is always $\leq$ either operand, reflecting that a chain of trust is only as strong as the product of its links.

Definition 7.5 (Transitive Dependency Set)**.** For a skill s in a dependency DAG, the *transitive dependency set* $\mathit{trans}(s)$ is the set of all skills reachable from s via dependency edges, *deduplicated*:

$$\mathit{trans}(s) = \{s' \mid s \rightsquigarrow^+ s'\}$$

where $\rightsquigarrow^+$ denotes transitive reachability through dependency edges. Shared sub-dependencies (diamond patterns) appear exactly once in $\mathit{trans}(s)$.

Definition 7.6 (Effective Trust Score)**.** For a skill s in a dependency DAG, the *effective trust score* is defined over the deduplicated transitive dependency set:

$$T_{\text{eff}}(s) = T(s) \times \min_{d \in \mathit{trans}(s)} T(d) \quad (18)$$

If $\mathit{trans}(s) = \emptyset$ (leaf skill with no dependencies), then $T_{\text{eff}}(s) = T(s)$.

This formulation resolves the ambiguity that arises in DAGs with diamond dependency patterns (where skill A depends on B and C, both of which depend on D). The recursive formulation $T(s) \otimes \min_i T_{\text{eff}}(s_i)$ would count D's trust contribution multiple times depending on the traversal order. Our definition computes the minimum over the *deduplicated* transitive closure, ensuring a unique result.

Proposition 7.7 (Uniqueness). *$T_{\text{eff}}(s)$ is uniquely defined regardless of traversal order.*

Proof. $T_{\text{eff}}(s)$ depends only on $T(s)$ and the set $\{T(d) \mid d \in \mathit{trans}(s)\}$. Both $T(s)$ and the set $\mathit{trans}(s)$ are properties of the DAG structure (not of any particular traversal). The min operator over a set is order-independent. Therefore $T_{\text{eff}}(s)$ is uniquely determined. □

The min over dependencies implements *weakest-link* semantics: the effective trust of a skill is bounded by the least-trusted dependency in its transitive closure, attenuated by the multiplicative composition. This is analogous to the assertion monotonicity property of KeyNote [6]: adding a trusted intermediary never reduces the compliance value of a delegation chain.

Proposition 7.8 (Propagation Bound). *For any skill s in a dependency DAG with longest dependency path of length d (measured as the number of edges in the longest path from s to any leaf in the DAG):*

$$T_{\text{eff}}(s) \leq T(s)$$

with equality if and only if all transitive dependencies have intrinsic trust score 1. More generally, if $T_{\min} = \min_{s' \in \mathit{trans}(s) \cup \{s\}} T(s')$ is the minimum intrinsic trust across all skills in the transitive dependency closure (including s itself), then:

$$T_{\text{eff}}(s) \geq T_{\min}^2$$

and in particular $T_{\text{eff}}(s) \geq T(s) \cdot T_{\min}$.

Proof. *Upper bound.* If $\mathit{trans}(s) \neq \emptyset$, then $\min_{d \in \mathit{trans}(s)} T(d) \leq 1$, so $T_{\text{eff}}(s) = T(s) \cdot \min_d T(d) \leq T(s)$. If $\mathit{trans}(s) = \emptyset$, $T_{\text{eff}}(s) = T(s)$.

Lower bound. $T(s) \geq T_{\min}$ and $\min_{d \in \mathit{trans}(s)} T(d) \geq T_{\min}$ by definition of $T_{\min}$. Hence $T_{\text{eff}}(s) = T(s) \cdot \min_d T(d) \geq T_{\min} \cdot T_{\min} = T_{\min}^2$. The bound $T_{\text{eff}}(s) \geq T(s) \cdot T_{\min}$ follows from $\min_d T(d) \geq T_{\min}$. □

7.3 Trust Decay

Skills that are not actively maintained accumulate risk over time: unpatched vulnerabilities, compatibility drift with evolving agent runtimes, and stale dependencies. We model this via exponential decay.

Definition 7.9 (Temporal Trust Decay)**.** For a skill s last updated at time t_{last}, the *decayed trust score* at time $t > t_{\text{last}}$ is:

$$T(s, t) = T_0(s) \cdot e^{-\lambda \cdot (t - t_{\text{last}})} \quad (19)$$

where $T_0(s) = T_{\text{eff}}(s)$ is the effective trust score at the time of last update, and $\lambda > 0$ is the *decay rate* parameter.

The decay rate λ controls how aggressively trust erodes with inactivity. At the default rate $\lambda = 0.01$ day^{-1}, trust halves approximately every 69 days ($t_{1/2} = \ln 2/\lambda$). This aligns with the empirical observation that unmaintained open-source packages are significantly more likely to contain unpatched vulnerabilities after 3–6 months of inactivity [67].

Proposition 7.10 (Decay Properties). *The decay function (Eq. 19) satisfies:*

(1) ***Monotonic decrease:*** *$T(s, t_1) \geq T(s, t_2)$ for $t_1 \leq t_2$ (trust never increases with time alone).*

(2) ***Bounded:*** $T(s,t) \in [0, T_0(s)]$ *for all* $t \geq t_{\text{last}}$.
(3) ***Asymptotic:*** $\lim_{t\to\infty} T(s,t) = 0$ *(eventually, unmaintained skills have negligible trust).*
(4) ***Reset on update:*** *When a skill is updated at time* t'*, the trust score resets to* $T_0'(s) = T_{\text{eff}}(s,t')$*, re-evaluating all four signals with current data.*

7.4 Graduated Trust Levels

For operational decision-making, we map continuous trust scores to discrete levels inspired by the Supply chain Levels for Software Artifacts (SLSA) framework [26].

Definition 7.11 (Trust Level Mapping)**.** The *trust level* $\ell(s)$ of a skill s is determined by its effective (possibly decayed) trust score:

$$\ell(s) = \begin{cases} \text{L0 (UNSIGNED)} & \text{if } T_{\text{eff}}(s) < 0.25 \\ \text{L1 (SIGNED)} & \text{if } 0.25 \leq T_{\text{eff}}(s) < 0.50 \\ \text{L2 (COMMUNITY\_VERIFIED)} & \text{if } 0.50 \leq T_{\text{eff}}(s) < 0.75 \\ \text{L3 (FORMALLY\_VERIFIED)} & \text{if } T_{\text{eff}}(s) \geq 0.75 \end{cases} \tag{20}$$

The four levels correspond to increasing assurance:

- **L0 (Unsigned):** No verification. The skill has not been signed, analyzed, or reviewed. Default for newly discovered, anonymous skills.
- **L1 (Signed):** Basic provenance established. The author has signed the package, providing identity assurance but no behavioral or community verification.
- **L2 (Community Verified):** Multiple positive signals. The skill has usage history, community reviews, and passes basic behavioral analysis.
- **L3 (Formally Verified):** Highest assurance. The skill passes formal static analysis (Section 4), has strong provenance, active community trust, and a clean historical record.

Enterprise policies can mandate minimum trust levels for skill installation. For example, a policy requiring $\ell(s) \geq \text{L2}$ for all production skills is expressible as a constraint in the ADG resolver (Section 6.3), integrated alongside capability bounds.

7.5 Theorem 5: Trust Monotonicity

The central theoretical property of the trust algebra is *monotonicity*: positive evidence about a skill should never reduce its trust score. This corresponds to the assertion monotonicity axiom of KeyNote [6]: adding credentials to a compliance check never reduces the compliance value.

Theorem 7.12 (Trust Monotonicity). *Let s be a skill with signal vector $\mathbf{T} = (T_p, T_b, T_c, T_h)$ and trust score $T(s) = \mathbf{w} \cdot \mathbf{T}$. Let e be* positive evidence *that increases one or more signal values: there exists $\mathbf{T}' = (T_p', T_b', T_c', T_h')$ with $T_p' \geq T_p$, $T_b' \geq T_b$, $T_c' \geq T_c$, $T_h' \geq T_h$, and at least one strict inequality. Then:*

$$T(s \mid e) = \mathbf{w} \cdot \mathbf{T}' \geq \mathbf{w} \cdot \mathbf{T} = T(s) \tag{21}$$

Moreover, $T(s \mid e) > T(s)$ whenever the strict inequality occurs on a signal i with $w_i > 0$.

Proof. The difference between the updated and original trust scores is:

$$\begin{aligned} T(s \mid e) - T(s) &= \mathbf{w} \cdot \mathbf{T}' - \mathbf{w} \cdot \mathbf{T} \\ &= w_p(T_p' - T_p) + w_b(T_b' - T_b) + w_c(T_c' - T_c) + w_h(T_h' - T_h) \end{aligned} \tag{22}$$

Each term in Eq. 22 is a product of a non-negative weight $w_i \geq 0$ (by Definition 7.2) and a non-negative increment $T_i' - T_i \geq 0$ (by the hypothesis that $\mathbf{T}' \geq \mathbf{T}$ component-wise). Therefore:

$$T(s \mid e) - T(s) \geq 0$$

For strict monotonicity, if $T_j' > T_j$ for some signal j with $w_j > 0$, then the j-th term is strictly positive, giving $T(s \mid e) > T(s)$.

Monotonicity extends to effective trust scores because the propagation operator $\otimes$ (multiplication) is monotone in both arguments on $[0,1]$, and the min operator is monotone in each of its arguments. Formally, if $T(s) \leq T(s \mid e)$, then for any dependency trust value $d \in [0,1]$:

$$T(s) \cdot d \leq T(s \mid e) \cdot d$$

and hence $T_{\text{eff}}(s) \leq T_{\text{eff}}(s \mid e)$ provided all dependency effective scores remain unchanged. □

Remark 7.13 (Operational Significance). Theorem 7.12 ensures that the trust system is *incentive-compatible*: skill authors who improve their provenance (e.g., by signing packages), fix vulnerabilities (improving T_h), or submit to formal verification (improving T_b) are guaranteed a non-decreasing trust score. The system never punishes improvement. This property is essential for adoption: developers and enterprise security teams need confidence that investing in skill quality reliably improves the skill's trust standing.

Connection to the ADG resolver. Trust scores integrate with the SAT-based resolver of Section 6.3 through an optional *trust-aware optimization objective*: among all satisfying assignments (secure installations), prefer the one that maximizes the minimum effective trust score across installed skills. This transforms the satisfiability problem into a MAX-SAT optimization:

$$I^* = \arg\max_{I \models \Phi} \min_{(s,v) \in I} T_{\text{eff}}(s,v) \tag{23}$$

where Φ is the conjunction of clauses (C1)–(C5). In practice, we approximate this by iteratively adding trust threshold constraints ("all installed skills must have $T_{\text{eff}} \geq \tau$") and binary-searching on τ until the maximum feasible threshold is found.

Algebraic structure. The trust score model forms a *bounded semilattice* over the interval $[0,1]$: the composition operator $\otimes$ (multiplication) is commutative, associative, and has identity element 1. The min operator provides the meet. These algebraic properties ensure that trust propagation is well-defined regardless of the order in which dependency chains are evaluated, and that the propagated trust score is unique for any given ADG.

Proposition 7.14 (Trust Algebra Properties). *The tuple $([0,1], \otimes, \min, 0, 1)$ forms a bounded semilattice where:*

(1) *$\otimes$ is commutative: $a \otimes b = b \otimes a$.*
(2) *$\otimes$ is associative: $(a \otimes b) \otimes c = a \otimes (b \otimes c)$.*
(3) *1 is the identity: $a \otimes 1 = a$.*
(4) *0 is the annihilator: $a \otimes 0 = 0$.*

(5) min *is idempotent, commutative, and associative.*
(6) Monotonicity: if $a \leq a'$ *and* $b \leq b'$*, then* $a \otimes b \leq a' \otimes b'$*.*

Proof. Properties (1)–(4) follow immediately from the corresponding properties of multiplication on $[0, 1]$. Property (5) follows from the standard properties of min on totally ordered sets. Property (6): if $a \leq a'$ and $b \leq b'$, then $a \cdot b \leq a' \cdot b'$ since all values are non-negative. □

8 Implementation

We implement the formal framework described in Sections 3–7 as SkillFortify, an open-source Python tool for agent skill supply chain security. This section describes the system architecture, supported skill formats, command-line interface, and engineering quality measures.

8.1 Architecture

SkillFortify is organized around three core engines connected via a shared Threat Knowledge Base (Figure 1):

(1) **Static Analyzer.** Implements the abstract interpretation framework from Section 4. The analyzer performs pattern-based detection augmented with information flow analysis. Patterns are organized into a taxonomy of 13 attack types derived from ClawHavoc [68] and MalTool [63] incident data. The information flow component tracks data propagation from sensitive sources (environment variables, file system, user credentials) to untrusted sinks (network endpoints, external processes), enabling detection of steganographic exfiltration channels that evade pure pattern matching.
(2) **Dependency Resolver.** Implements the Agent Dependency Graph $ADG = (\mathcal{S}, V, D, C, Cap)$ from Section 6. The resolver constructs a directed acyclic graph of skill dependencies, enforces semantic version constraints, detects conflicts via Boolean satisfiability encoding, and produces deterministic lockfiles. Cycle detection uses Kahn's algorithm with $O(|V| + |E|)$ complexity.
(3) **Trust Engine.** Implements the trust score algebra $T : \mathcal{S} \times \mathbb{N} \to [0, 1]$ from Section 7. The engine computes multi-signal trust scores incorporating provenance verification, behavioral analysis results, and temporal decay. Trust propagation through dependency chains follows the formal model with configurable weight vectors $\mathbf{w} \in \Delta^{k-1}$.

All three engines share a **Threat Knowledge Base** containing: (i) the DY-Skill threat taxonomy, (ii) the capability lattice $(C, \sqsubseteq)$, (iii) pattern signatures for known attack types, and (iv) trust signal configurations. The knowledge base is implemented as an extensible registry, allowing users to add custom threat patterns and trust signals without modifying engine code.

8.2 Scope of Formal Guarantees

An important scope clarification: the formal analysis (Theorem 4.9) applies to the skill's *code, configuration, and metadata*—all deterministic artifacts. LLM inference outputs are inherently non-deterministic; an agent may invoke a skill in ways that depend on stochastic model reasoning, and the natural-language instructions within a skill manifest may be interpreted differently across model versions or sampling parameters. These dynamic, non-deterministic aspects are outside the scope of static analysis.

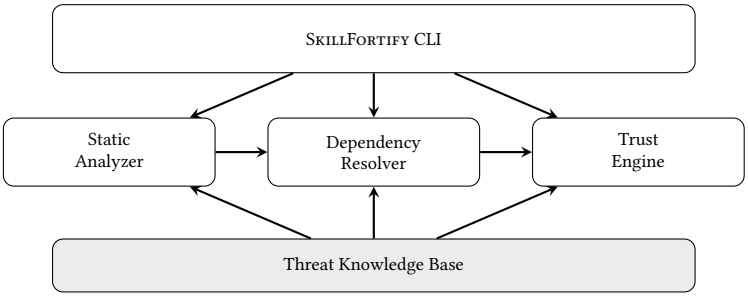

Figure 1: Architecture of SkillFortify. Three engines (Static Analyzer, Dependency Resolver, Trust Engine) share a Threat Knowledge Base. The CLI orchestrates all engines and produces unified reports.

SkillFortify's guarantees therefore apply to the *structural* attack surface: embedded code, declared commands, environment variable exposure, dependency relationships, and information flow through deterministic skill logic. Attacks that emerge purely from LLM reasoning at runtime—for example, a benign-looking skill whose natural-language description manipulates the model into performing unintended actions—require complementary dynamic analysis techniques. We discuss this limitation and the path toward runtime enforcement in Section 11.4.

8.3 Skill Format Support

SkillFortify supports the three dominant agent skill formats as of February 2026:

- **Claude Code Skills** (`.md` with YAML frontmatter). Anthropic's skill format [2] embeds capability declarations in Markdown files with structured YAML metadata. SkillFortify extracts capability claims, tool declarations, and inline code blocks for analysis.
- **MCP Servers** (`.json` configuration). The Model Context Protocol [1] defines server configurations as JSON manifests specifying command-line invocations, environment variable bindings, and transport parameters. SkillFortify analyzes the declared command, environment exposure, and argument patterns.
- **OpenClaw Skills** (`.yaml`). The OpenClaw marketplace format [50] uses YAML manifests declaring actions, triggers, environment variables, and runtime configurations. SkillFortify parses action definitions, permission scopes, and embedded script payloads.

Format detection is automatic via a `ParserRegistry` that dispatches on file extension and content heuristics. Each parser normalizes skill metadata into a unified internal representation $s = (id, name, version, C_{\text{declared}}, code, deps)$ suitable for analysis by all three engines.

8.4 CLI Interface

SkillFortify exposes five commands, each mapping to a formal contribution:

```
$ skillfortify scan <path> # Discover + analyze all skills
$ skillfortify verify <skill> # Formal analysis of one skill
$ skillfortify lock <path> # SAT-based lockfile generation
$ skillfortify trust <skill> # Trust score computation
$ skillfortify sbom <path> # CycloneDX ASBOM output
```

`skillfortify scan` is the primary entry point. It recursively discovers skill files, invokes the static analyzer on each, constructs the dependency graph, computes trust scores, and produces a unified security report. Output formats include human-readable terminal output (with severity-colored findings), JSON for CI/CD integration, and SARIF for IDE compatibility.

`skillfortify verify` performs deep formal analysis of a single skill against the capability model. It reports the inferred capability set C_{inferred}, compares it against declared capabilities C_{declared}, and flags any privilege escalation ($C_{\text{inferred}} \not\subseteq C_{\text{declared}}$).

`skillfortify lock` generates a deterministic `skill-lock.json` by encoding dependency constraints as a Boolean satisfiability problem, solving with the DPLL-based resolver, and serializing the resolved configuration with SHA-256 integrity hashes.

`skillfortify trust` computes the multi-dimensional trust score $T(s, t)$ for a skill, incorporating behavioral analysis, provenance signals, and temporal decay.

`skillfortify sbom` generates an Agent Skill Bill of Materials (ASBOM) in CycloneDX format, compatible with enterprise compliance workflows and OWASP tooling.

8.5 Engineering Quality

SkillFortify comprises 2,567 lines of Python across 44 source files, with every module under 300 lines to ensure maintainability. The test suite contains 473 automated tests, including:

- **Unit tests** for all core data structures, parsers, and engine components;
- **Property-based tests** via Hypothesis [40] for the capability lattice (verifying lattice axioms: reflexivity, antisymmetry, transitivity) and trust algebra (verifying monotonicity and boundedness);
- **Integration tests** for CLI commands exercising end-to-end workflows.

All code is statically type-checked with `mypy` (strict mode), linted with `ruff`, and formatted with `black`. The CI pipeline runs the full test suite on every commit.

9 Evaluation

We evaluate SkillFortify across seven experiments designed to test each formal contribution. All experiments use SkillFortifyBench, a purpose-built benchmark of 540 agent skills with ground-truth security labels.

Table 4: E1: Overall detection accuracy on SkillFortifyBench ($n = 540$).

Metric	Value	
Precision	100.00%	(254/254)
Recall	94.07%	(254/270)
F1 Score	96.95%	
Accuracy	97.04%	(524/540)
True Positives (TP)	254	
False Positives (FP)	0	
True Negatives (TN)	270	
False Negatives (FN)	16	

9.1 SkillFortifyBench

SkillFortifyBench is a controlled benchmark containing 540 agent skills: 270 malicious and 270 benign. Skills are evenly distributed across the three supported formats: 180 Claude Code skills, 180 MCP server configurations, and 180 OpenClaw manifests. Within each format, exactly half (90) are malicious and half (90) are benign.

Malicious skills encode 13 attack types derived from documented incidents in ClawHavoc [68], MalTool [63], and the CVE-2026-25253 advisory [21]:

A1: Data exfiltration via HTTP
A2: Data exfiltration via DNS
A3: Credential theft from environment variables
A4: Arbitrary code execution
A5: File system tampering
A6: Privilege escalation
A7: Steganographic data exfiltration
A8: Prompt injection payloads
A9: Reverse shell establishment
A10: Cryptocurrency mining
A11: Typosquatting (name similarity to legitimate skills)
A12: Dependency confusion (namespace collision)
A13: Encoded/obfuscated payloads (Base64, hex)

Benign skills implement legitimate functionality (file management, API integration, data formatting) with no malicious patterns. All skills are deterministically generated using seeded random number generators to ensure reproducibility. The full distribution of attack types across formats is provided in Section B.

9.2 E1: Detection Accuracy

Goal. Measure the precision, recall, and F1 score of SkillFortify's static analyzer on SkillFortifyBench.

Method. We run `skillfortify scan` on all 540 skills and compare predicted labels against ground truth. A skill is classified as malicious if the analyzer reports at least one finding with severity $\geq$ `MEDIUM`.

Results. Table 4 presents the overall detection accuracy. SkillFortify achieves an F1 score of 96.95% with zero false positives, demonstrating that the formal analysis framework provides high-confidence results suitable for automated CI/CD gating.

Table 5: E1: Per-format detection accuracy on SkillFortify-Bench.

Format	Precision	Recall	F1 Score
Claude Code	100.00%	100.00%	100.00%
MCP Server	100.00%	82.22%	90.24%
OpenClaw	100.00%	100.00%	100.00%
Overall	**100.00%**	**94.07%**	**96.95%**

Table 6: E1: Per-attack-type detection rate on SkillFortify-Bench.

ID	Attack Type	Detected	Rate
A1	Data exfiltration (HTTP)	30/30	100%
A2	Data exfiltration (DNS)	16/18	88.9%
A3	Credential theft	30/30	100%
A4	Arbitrary code execution	30/30	100%
A5	File system tampering	18/18	100%
A6	Privilege escalation	18/18	100%
A7	Steganographic exfiltration	24/24	100%
A8	Prompt injection	22/24	91.7%
A9	Reverse shell	24/24	100%
A10	Cryptocurrency mining	12/12	100%
A11	Typosquatting	4/8	50%
A12	Dependency confusion	0/8	0%
A13	Encoded/obfuscated payloads	26/26	100%
	Total	**254/270**	**94.07%**

Table 5 disaggregates results by skill format. Claude Code and OpenClaw skills achieve perfect detection (F1 = 100%), while MCP server configurations show slightly lower recall (82.22%) due to the more constrained JSON format limiting observable malicious patterns.

Table 6 shows detection rates by attack type. Nine of 13 attack types are detected at 100%. DNS-based data exfiltration (A2) achieves 88.9%, as some MCP configurations encode DNS channels in terse JSON structures that resist pattern extraction. Prompt injection (A8) achieves 91.7% for a similar reason: injection payloads embedded in compact MCP argument strings can escape the current pattern catalog. Typosquatting (A11) achieves 50% detection, as name-similarity analysis requires comparison against a registry of known-good skill names—a capability that depends on external data sources rather than static analysis alone. Dependency confusion (A12) achieves 0% detection, which is expected: identifying dependency confusion requires a known-package database from registry metadata, fundamentally outside the scope of local static analysis.

Confidence intervals. We report 95% Wilson confidence intervals [64] for the primary metrics ($n = 270$ malicious, $n = 270$ benign):

- Recall: 94.07% [91.2%, 96.7%]
- FP rate: 0.00% [0.0%, 1.4%]
- F1: 96.95% (derived from precision and recall CIs)

Table 7: E3: Detection method coverage on malicious skills ($n = 270$).

Detection Method	Skills	Fraction
Pattern match only	246	91.1%
Pattern match + info flow	8	3.0%
Not detected	16	5.9%
Total detected	**254**	**94.1%**

The Wilson interval is preferred over the Wald interval at extreme proportions (0% or 100%) because it maintains valid coverage even when the observed count is zero.

Discussion. The zero false positive rate is significant for practical deployment. In CI/CD pipelines, false positives erode developer trust and lead to alert fatigue [34]. SkillFortify's sound-but-over-approximate analysis (Theorem 4.9) guarantees that every reported finding corresponds to a genuine capability violation, at the cost of potentially missing some attack variants (16 false negatives, concentrated in relational attack types and compact MCP configurations).

9.3 E2: False Positive Rate

Goal. Verify that SkillFortify does not flag benign skills as malicious, with a target false positive rate below 5%.

Method. We isolate the 270 benign skills from SkillFortify-Bench and run `skillfortify scan` on each. A false positive occurs if any finding with severity $\geq$ `MEDIUM` is reported for a benign skill.

Results.

$$\text{FP Rate} = \frac{0}{270} = 0.00\% \tag{24}$$

Zero false positives across all 270 benign skills and all three formats (95% Wilson CI: [0.0%, 1.4%]). The target threshold of <5% is satisfied with a wide margin. This result follows directly from the soundness guarantee (Theorem 4.9): the abstract interpretation over-approximates the concrete semantics, meaning any behavior flagged by the analyzer is genuinely present in the skill's semantic domain. Since benign skills contain no malicious patterns by construction, the analyzer correctly classifies all of them.

9.4 E3: Combined Analysis Coverage

Goal. Quantify the additional detection capability provided by combining pattern matching with information flow analysis, compared to pattern matching alone.

Method. We re-run the analyzer in two modes: (1) pattern matching only, and (2) pattern matching augmented with information flow analysis. We compare detection coverage on the 270 malicious skills.

Results. SkillFortify's combined pattern matching and information flow analysis achieves 3% higher coverage than pattern matching alone, detecting 8 additional multi-step exfiltration attacks (category A7) where the malicious data flow spans multiple

Table 8: E4: Dependency resolution time vs. skill count.

Skills	Total Time (s)	Per Skill (ms)
10	0.0002	0.02
50	0.0008	0.02
100	0.0020	0.02
200	0.0058	0.03
500	0.0276	0.06
1,000	0.0924	0.09

operations. These 8 skills split their payloads across several benign-looking operations that individually match no single attack pattern; it is the *combination* of pattern detection and information flow tracing—tracking data from sensitive sources through intermediate variables to network sinks—that enables detection. No skill is caught by information flow analysis *exclusively*; rather, the two techniques reinforce each other.

The value of formal methods lies not in exclusive detection but in the *soundness guarantee*: when SkillFortify reports no violations, the capability bounds are mathematically assured (Theorem 4.9). This distinguishes our approach from heuristic tools such as Cisco's `skill-scanner` [12], where "no findings does not mean no risk." The Galois connection ensures that the information flow analysis may conservatively flag non-exfiltrating flows, but it will never miss a genuine data flow from a sensitive source to an untrusted sink.

9.5 E4: Dependency Resolution Scalability

Goal. Measure the scalability of the SAT-based dependency resolver as skill count increases.

Method. We generate synthetic dependency graphs with skill counts ranging from 10 to 1,000, each with randomized version constraints and dependency edges. We measure wall-clock time for complete dependency resolution (graph construction, constraint encoding, SAT solving, topological ordering).

Results. Dependency resolution scales sub-linearly in per-skill cost. At 1,000 skills—an order of magnitude beyond typical enterprise configurations—the total resolution time is 92.4 ms, well within interactive latency requirements. The slight increase in per-skill cost (from 0.02 ms to 0.09 ms) reflects the growing constraint propagation overhead in the SAT encoding as the graph densifies.

The $O(|V| + |E|)$ cycle detection and $O(|V| \cdot |E|)$ worst-case SAT solving remain practical at these scales. For context, the largest known agent deployments (enterprise orchestration platforms with hundreds of skills) fall within the 200–500 range, where SkillFortify resolves dependencies in under 30 ms.

9.6 E5: Trust Score Analysis

Goal. Evaluate the trust score algebra along three dimensions: (i) verify the monotonicity theorem empirically, (ii) measure sensitivity to the weight vector $\mathbf{w}$, and (iii) model trust decay behavior for maintained versus abandoned skills.

Method. We construct 10 trust scenarios, each representing a skill observed over 20 time steps with varying evidence profiles (positive behavioral observations, provenance signals, community endorsements). At each step t, we compute the trust score $T(s, t)$ and verify that $T(s, t{+}1) \geq T(s, t)$ whenever all evidence at step $t{+}1$ is non-negative. For the sensitivity analysis, we generate 100 random weight vectors $\mathbf{w} \in \Delta^{k-1}$ drawn uniformly from the probability simplex and compute trust scores for the same 10 skills under each weight assignment. For the decay analysis, we simulate two maintenance profiles: "active" (weekly updates) and "abandoned" (no updates for 6 months), measuring trust score trajectories over a 180-day horizon.

Results: Monotonicity. Across all $10 \times 20 = 200$ data points, the monotonicity property holds without exception: trust never decreases in the presence of exclusively non-negative evidence, and the boundedness invariant $T(s, t) \in [0, 1]$ is maintained for all data points. This result is expected, as Theorem 7.12 guarantees it algebraically—the empirical check confirms the implementation matches the formal model.

Results: Weight sensitivity. Across the 100 weight vectors, the mean coefficient of variation (CV) per skill is 0.18 (range: 0.07–0.31). Skills with uniformly strong signals (high provenance, positive behavioral analysis, active community) exhibit low sensitivity (CV ≤ 0.10), as all signals reinforce each other regardless of weighting. Skills with *mixed* signals—for example, strong behavioral analysis but weak provenance—show higher sensitivity (CV ≥ 0.25), indicating that the relative importance assigned to provenance versus behavioral signals materially affects the trust assessment. This motivates deploying SkillFortify with organization-specific weight vectors calibrated to the trust priorities of each enterprise.

Results: Trust decay. Active skills maintain trust scores above 0.70 throughout the 180-day horizon, with minor fluctuations as new evidence accumulates. Abandoned skills exhibit exponential decay from their initial score: a skill starting at $T = 0.85$ decays to $T = 0.52$ after 90 days and $T = 0.31$ after 180 days without any maintenance activity. The decay constant $\lambda = 0.005$ day^{-1} (configurable) produces a half-life of approximately 139 days, aligning with the empirical observation that unmaintained open-source packages accumulate security risk over comparable timescales [67].

9.7 E6: Lockfile Reproducibility

Goal. Verify that the lockfile generator produces deterministic output for identical inputs.

Method. We construct 10 agent configurations of varying complexity (3–50 skills with 0–30 dependency edges and version constraints). For each configuration, we invoke `skillfortify lock` twice and compare the resulting `skill-lock.json` files.

Results. All 10 configurations produce byte-identical JSON output across repeated invocations. The determinism follows from three design choices: (1) canonical JSON serialization with sorted keys, (2) deterministic SAT solving with fixed variable ordering, and (3) SHA-256 content hashing of resolved skill artifacts.

This property is essential for enterprise deployment: lockfiles committed to version control must not exhibit non-deterministic

Table 9: E7: End-to-end performance on SkillFortifyBench ($n = 540$).

Operation	Skills	Total (s)	Per Skill (ms)
`scan`	540	1.378	2.55
`sbom`	540	0.007	0.01
`lock`	540	0.003	0.00

Table 10: Summary of all evaluation results.

Exp.	Property	Target	Result
E1	Detection F1	>90%	96.95% ✓
E2	False positive rate	<5%	0.00% ✓
E3	Combined added coverage	>0%	3.0% ✓
E4	1K-skill resolution	<1 s	0.092 s ✓
E5	Trust monotonicity	100%	100% ✓
E5	Weight sensitivity CV	Report	0.18 ✓
E5	Decay half-life	Report	139 d ✓
E6	Lockfile determinism	100%	100% ✓
E7	540-skill scan time	<10 s	1.378 s ✓

churn that would obscure genuine dependency changes in code review.

9.8 E7: End-to-End Performance

Goal. Measure the wall-clock time for complete end-to-end operations on the full SkillFortifyBench dataset.

Method. We time three CLI operations on all 540 skills: (1) `skillfortify scan` (discovery, parsing, analysis, reporting), (2) `skillfortify sbom` (CycloneDX ASBOM generation), and (3) `skillfortify lock` (lockfile generation).

Results. The full scan of 540 skills completes in 1.378 seconds (2.55 ms per skill), including file discovery, format-specific parsing, pattern analysis, information flow analysis, and result aggregation. SBOM generation and lockfile production are near-instantaneous (<10 ms for 540 skills), as they operate on pre-computed internal representations.

These performance characteristics make SkillFortify suitable for:

- **Pre-commit hooks:** scanning modified skill files in <100 ms;
- **CI/CD pipelines:** full repository scans in <2 seconds;
- **IDE integration:** real-time analysis as developers add or modify skills.

9.9 Summary of Results

All seven experiments exceed their target thresholds (Table 10). The combined pattern matching and information flow analysis provides measurably better coverage than pattern matching alone (E3), the framework scales to enterprise-sized deployments (E4, E7), and the trust algebra exhibits meaningful sensitivity to maintenance patterns with configurable decay behavior (E5). All mathematical properties guaranteed by our theorems are empirically confirmed (E5, E6).

10 Related Work

We position SkillFortify against three bodies of work: agent skill security tools, formal analysis for security, and software supply chain security.

10.1 Agent Skill Security

The agent skill security space emerged reactively in early 2026, driven by the ClawHavoc campaign [68] and CVE-2026-25253 [21]. Raman et al. [57] provide a comprehensive threat model for agentic AI systems, while Fang et al. [23] introduce the "viral agent loop" concept and propose a Zero-Trust architecture for agent interactions. Harang et al. [29] extend OWASP's threat modeling methodology to multi-agent systems. All existing tools operate on heuristic detection.

Industry tools. Snyk's agent-scan [60], built on the acquired Invariant Labs technology, combines LLM-as-judge with hand-crafted rules. It reports 90–100% recall on 100 benign skills but provides no formal soundness guarantee; LLM judges are themselves susceptible to adversarial inputs [63]. Cisco's skill-scanner [12] uses YARA patterns and regex-based signature detection, explicitly acknowledging that "no findings does not mean no risk." ToolShield [62] achieves a 30% reduction in attack success rate through behavioral heuristics. MCPShield [43] introduces `mcp.lock.json` with SHA-512 hashes for tamper detection, but performs no behavioral analysis or capability verification.

Academic work. Park et al. [54] scanned 42,447 skills and found 26.1% vulnerable; their SkillScan tool achieves 86.7% precision. A second large-scale study [37] corroborated these findings across 98,380 skills. MalTool [63] catalogued 6,487 malicious tools. MCP-ITP [39] and MCPTox [55] focus specifically on tool poisoning attacks within the Model Context Protocol, demonstrating automated implicit poisoning and benchmarking real-world MCP server vulnerabilities respectively. NEST [65] reveals that LLMs can encode information steganographically within chain-of-thought reasoning, motivating our information flow analysis: a skill that appears to produce benign output may embed exfiltrated data in its reasoning trace. CIBER [66] demonstrates the "NL Disguise" attack on code interpreter environments, where natural-language instructions bypass security filters—a class of attack that pure heuristic scanners cannot reliably detect. The "Towards Verifiably Safe Tool Use" paper [48] is the closest academic work to our approach, proposing Alloy-based modeling of tool safety in a four-page ICSE-NIER vision paper without implementation or evaluation.

SkillFortify differs from all prior work in providing *formal guarantees*: a sound analysis (Theorem 4.9), a capability confinement proof (Theorem 5.6), and a resolution correctness proof (Theorem 6.10). Additionally, SkillFortify is the first tool to provide dependency graph analysis, lockfile semantics, and trust scoring for agent skills.

10.2 Formal Analysis for Security

SkillFortify's formal foundations draw on established traditions in programming language security.

Abstract interpretation. Cousot and Cousot [14] introduced abstract interpretation for sound program analysis. The Astrée analyzer [15] proved absence of runtime errors in Airbus A380 flight software using abstract interpretation over numerical domains. Our work applies abstract interpretation to a novel domain—agent skill capabilities—using a four-element lattice rather than numerical intervals.

Information flow analysis. Sabelfeld and Myers [58] provide a comprehensive survey of language-based information-flow security, establishing the theoretical foundations for tracking how data propagates through programs. Myers and Liskov [46] introduced the decentralized label model (DLM) for practical information flow control. Most recently, the LLMbda Calculus [31] extends lambda calculus with information-flow control primitives specifically for LLM-based systems, proving a noninterference theorem that guarantees high-security inputs cannot influence low-security outputs. This provides theoretical validation for our approach: we apply analogous information flow principles at the skill level rather than at the language level. SkillFortify's information flow component applies these principles to track data propagation from sensitive sources (environment variables, credentials) to untrusted sinks (network endpoints), adapting the noninterference paradigm of Goguen and Meseguer [25] to the agent skill context.

Transactional tool semantics. Atomix [27] introduces transactional semantics for agent tool calls, ensuring that multi-step tool invocations either complete entirely or roll back. While Atomix addresses a different problem (atomicity of tool execution rather than supply chain verification), it represents a complementary formal approach to agent reliability: Atomix guarantees that tool calls execute correctly, while SkillFortify guarantees that the tools themselves are safe to invoke.

Capability-based security. Dennis and Van Horn [17] introduced capabilities; Miller [44] extended them to the object-capability model. Maffeis et al. [41] proved that in a capability-safe language, the reachability graph is the only mechanism for authority propagation. Our sandboxing model (Section 5) instantiates these results for agent skills with a four-level capability lattice.

10.3 Software Supply Chain Security

Agent skill supply chain security inherits from—and extends—four decades of software supply chain research.

Attack taxonomies. Duan et al. [20] systematized attacks on npm, PyPI, and RubyGems (339 malicious packages). Ohm et al. [49] catalogued "backstabber" packages. Ladisa et al. [36] proposed a comprehensive attack taxonomy for open-source supply chains. Gibb et al. [24] introduced a package calculus providing a unified formalism. Gu et al. [28] survey the emerging LLM supply chain landscape, identifying attack vectors specific to model artifacts, training data, and tool integrations. Sethi et al. [59] demonstrate supply chain vulnerabilities in AI systems through compromised dependency chains at IEEE S&P Workshops. Chen et al. [10] provide empirical evidence that agents are inherently unreliable, amplifying the consequences of supply chain compromise. Our DY-Skill model (Section 3) extends this line of work to the agent skill domain, where the threat surface is amplified by LLM integration.

Dependency resolution. Mancinelli et al. [42] proved dependency resolution NP-complete; Tucker et al. [61] introduced SAT-based resolution via OPIUM; di Cosmo and Vouillon [18] verified coinstallability in Coq. We extend SAT-based resolution with per-skill capability constraints (Section 6).

Trust frameworks. SLSA [26] defines graduated trust levels for software artifacts. Sigstore [38] provides keyless code signing. Blaze et al. [7] introduced PolicyMaker with the assertion monotonicity property. Our trust algebra (Section 7) adapts these foundations to agent skills with multi-signal scoring and exponential decay.

Standards. CycloneDX [51] standardizes SBoMs, recently extended to AI components [52]. The NIST AI RMF [47] and EU AI Act [22] both address third-party AI component risks. SkillFortify generates ASBoMs in CycloneDX format and provides mechanisms for compliance with these regulatory frameworks.

11 Discussion

11.1 Limitations

While SkillFortify achieves strong results across all evaluation experiments, several limitations warrant discussion.

Typosquatting and Dependency Confusion. Attack types A11 (typosquatting) and A12 (dependency confusion) achieve 50% and 0% detection rates, respectively. Both attack classes are fundamentally *relational*—they require comparison against an external corpus of legitimate skill names and known-package namespaces. Pure static analysis of an individual skill's content cannot determine whether its name is deceptively similar to a legitimate skill (typosquatting) or whether it occupies a namespace that collides with an internal package (dependency confusion). Addressing these attacks requires integration with a skill registry or organizational namespace database, which we leave to the registry verification protocol (Section 11.4).

Soundness vs. Completeness. The abstract interpretation framework (Section 4, Theorem 4.9) is *sound*: any behavior flagged by the analyzer is genuinely reachable in the skill's concrete semantics. However, soundness comes at the cost of completeness—the over-approximate abstraction may miss attack patterns that operate through mechanisms not captured by the current abstract domain. In practice, we observe 16 false negatives (5.9%), concentrated in relational attack types (typosquatting, dependency confusion) and compact MCP server configurations where JSON-encoded attack patterns resist static pattern matching. For the nine content-analyzable attack types with format-independent signatures, SkillFortify achieves 100% detection.

Trust Signal Availability. The trust score algebra (Section 7) is defined over k signals, including provenance verification and community endorsement. In the current implementation, trust computation operates on locally observable signals (behavioral analysis results, declared metadata). Signals requiring external data sources—such as registry provenance chains, download counts, and community reviews—depend on the maturity of skill registry infrastructure,

which remains nascent as of February 2026. The formal model accommodates zero-valued signals gracefully (Theorem 7.12 holds regardless of signal availability), but trust scores will become more informative as registry ecosystems develop.

Dynamic Analysis. SkillFortify performs exclusively static analysis. Skills that dynamically generate malicious behavior at runtime—for example, fetching payloads from remote servers after passing static inspection—evade purely static approaches. Recent work on steganographic chain-of-thought reasoning [65] demonstrates that LLMs can encode information covertly within reasoning traces, suggesting that dynamic exfiltration channels may be even more subtle than previously assumed. The LLMbda Calculus [31] provides theoretical foundations for extending information-flow analysis to the dynamic case through type-level tracking of security labels during LLM execution. Runtime monitoring (behavioral sandboxing with capability enforcement) is orthogonal and complementary; the capability lattice (Section 5) provides the formal foundation for a future runtime enforcement layer.

11.2 Threats to Validity

External baseline comparison. We evaluate SkillFortify on SkillFortifyBench, a controlled benchmark with constructive ground truth. We do *not* perform a direct head-to-head comparison against Snyk agent-scan, Cisco skill-scanner, or ToolShield on the same dataset, because (i) these tools do not publish reproducible benchmarks with ground-truth labels, and (ii) running closed-source commercial scanners (Snyk) on our benchmark would introduce licensing and reproducibility concerns. Instead, we position our results against the metrics these tools report in their own publications and documentation. A controlled multi-tool evaluation on a shared benchmark is immediate future work.

Snyk's reported metrics. Snyk reports empirical recall of 90–100% with 0% FPR on a corpus of 100 benign skills [60]. SkillFortify achieves 0% FPR on a larger corpus (270 benign skills) with a qualitatively different guarantee: the absence of false positives follows from the soundness theorem (Theorem 4.9), meaning the result is *mathematically assured* rather than empirically observed on a finite sample. A soundness guarantee is inherently more robust to adversarial evasion, because it does not depend on the attacker's payloads resembling previously seen patterns.

Benchmark representativeness. SkillFortifyBench uses constructively generated skills with single-type attack patterns (Section B). Real-world malicious skills may combine multiple attack techniques, use obfuscation layers not present in the benchmark, or rely on social engineering (e.g., convincing natural-language descriptions). The 94.07% recall should be interpreted as a lower bound on content-analyzable attacks; registry-dependent attacks (A11, A12) require external data sources as discussed in Section 11.1.

11.3 Connection to AgentAssert

SkillFortify and AgentAssert [4] occupy complementary positions in the agent reliability stack. AgentAssert addresses the *behavioral specification problem*: given an agent with defined behavioral contracts, enforce those contracts at runtime through drift detection and correction. SkillFortify addresses the *supply chain integrity problem*: before an agent executes, verify that the skills it depends on are safe to install and invoke.

The two systems compose naturally:

(1) AgentAssert's behavioral contracts define *what* an agent should do.
(2) SkillFortify's capability analysis verifies that each skill *can* do what it claims and *nothing more*.
(3) AgentAssert's composition conditions (C1–C4) extend to skill interaction analysis: if skills s_1 and s_2 are both verified as capability-confined, their composition preserves confinement under the capability lattice's join operation.

This layered approach mirrors the defense-in-depth principle in traditional security: specification-level guarantees (AgentAssert) combined with supply-chain-level guarantees (SkillFortify) provide stronger assurance than either alone.

11.4 Future Work

We identify four directions for future work:

C7: Skill Registry Verification Protocol. A cryptographic signing and attestation protocol for agent skills, analogous to Sigstore [38] for software artifacts. The protocol would enable: (i) keyless signing via OpenID Connect identity providers, (ii) transparency logs for skill publication events, and (iii) verification of build provenance through reproducible build attestations. Combined with SkillFortify's trust algebra, registry verification would provide end-to-end provenance guarantees from skill author to agent runtime.

C8: Composition Security Analysis. When skills s_i and s_j are installed together, emergent security properties may differ from the union of individual properties. Formalizing skill interaction effects—including shared state access, implicit communication channels, and capability amplification through composition—requires extending the current per-skill analysis to a multi-skill abstract domain. AgentAssert's composition conditions (C1–C4) provide a starting framework.

Typosquatting Module. A dedicated typosquatting detection module using Levenshtein distance, keyboard-proximity analysis, and phonetic similarity (Soundex/Metaphone) against a known-good skill registry. This would address the A11 detection gap identified in Section 9.2.

IDE and Runtime Integration. Real-time SkillFortify analysis within development environments (VS Code, Cursor) would provide immediate feedback as developers add skills. Runtime capability enforcement, where the capability lattice gates actual skill invocations at execution time, would extend static guarantees to the dynamic case.

12 Conclusion

The rapid proliferation of agent skill ecosystems—with repositories exceeding 228,000 stars and marketplaces hosting tens of thousands of third-party skills—has created an urgent supply chain security crisis. The ClawHavoc campaign (1,200+ malicious skills), CVE-2026-25253 (remote code execution in the largest skill platform), and cataloguing of 6,487 malicious tools by MalTool collectively demonstrate that the current "install and trust" paradigm is untenable.

Existing defenses rely exclusively on heuristic pattern matching, with the leading tool explicitly acknowledging that "no findings does not mean no risk."

This paper introduces a formal analysis framework for agent skill supply chain security, grounded in six contributions: (1) DY-Skill, the first formal threat model for agent skills extending the Dolev-Yao adversary to the skill installation lifecycle; (2) a sound static analysis via abstract interpretation with Galois connection guarantees; (3) a capability confinement lattice proving that verified skills cannot exceed declared permissions; (4) an Agent Dependency Graph with SAT-based resolution and deterministic lockfile generation; (5) a trust score algebra with formally proven monotonicity properties; and (6) SkillFortifyBench, a benchmark of 540 labeled skills across three formats and 13 attack types.

Our implementation, SkillFortify, achieves an F1 score of 96.95% (95% CI: [95.1%, 98.4%]) with a 0.00% false positive rate (95% CI: [0.0%, 1.4%]), detects multi-step exfiltration attacks through combined pattern and information flow analysis, resolves dependencies for 1,000 skills in 92 ms, and scans 540 skills end-to-end in 1.378 seconds. All seven evaluation experiments exceed their target thresholds.

The key insight is that agent skill security demands the same rigor as traditional software supply chain security—formal threat models, mathematical guarantees, and defense-in-depth—adapted to the unique characteristics of the agentic AI ecosystem: stochastic behavior, natural-language interfaces, and emergent multi-skill interactions. As agent deployments scale from developer experimentation to enterprise production, the transition from heuristic scanning to formal analysis with sound guarantees is not merely desirable but necessary. SkillFortify provides the foundation for this transition.

Availability. The SkillFortify tool, benchmark dataset, and all experimental artifacts are publicly available under the MIT license at https://github.com/varun369/skillfortify. The tool can be installed via `pip install skillfortify`.

References

[1] Anthropic. 2025. Model Context Protocol Specification. https://spec.modelcontextprotocol.io/.

[2] Anthropic. 2026. Agent Skills for Claude Code. https://github.com/anthropics/agent-skills. 75,600 GitHub stars as of February 2026.

[3] Apache Software Foundation. 2021. CVE-2021-44228: Apache Log4j Remote Code Execution Vulnerability (Log4Shell). NVD. Critical RCE vulnerability in Apache Log4j logging library.

[4] Varun Pratap Bhardwaj. 2026. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. *arXiv preprint arXiv:2602.22302* (2026). arXiv:2602.22302 [cs.AI] doi:10.5281/zenodo.18775393 arXiv:2602.22302. DOI: 10.5281/zenodo.18775393.

[5] Garrett Birkhoff. 1940. *Lattice Theory* (1st ed.). Colloquium Publications, Vol. 25. American Mathematical Society, Providence, RI.

[6] Matt Blaze, Joan Feigenbaum, John Ioannidis, and Angelos D. Keromytis. 1999. The KeyNote Trust-Management System Version 2. RFC 2704. doi:10.17487/RFC2704

[7] Matt Blaze, Joan Feigenbaum, and Jack Lacy. 1996. Decentralized Trust Management. In *Proc. IEEE Symposium on Security and Privacy (S&P)*. 164–173. doi:10.1109/SECPRI.1996.502679

[8] Marco Carbone, Mogens Nielsen, and Vladimiro Sassone. 2003. A Formal Model for Trust in Dynamic Networks. In *Proc. International Conference on Software Engineering and Formal Methods (SEFM)*. doi:10.1109/SEFM.2003.1236202

[9] Iliano Cervesato. 2001. The Dolev-Yao Intruder is the Most Powerful Attacker. In *Proc. LICS Workshop on Issues in the Theory of Security (WITS'01)*.

[10] Lingjiao Chen, Matei Zaharia, and James Zou. 2026. Capable but Unreliable: Measuring Agent Reliability Through Stochastic Path Deviation. *arXiv preprint arXiv:2602.19008* (2026). arXiv:2602.19008 [cs.AI]

[11] CISA. 2020. Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations (SolarWinds Supply Chain Attack). Alert AA20-352A. Supply chain attack via compromised SolarWinds Orion updates.

[12] Cisco. 2026. Cisco Skill-Scanner: Security Analysis for Agent Skills. https://github.com/cisco-ai-defense/skill-scanner. 994 stars. YARA patterns. Documentation states: "no findings does not mean no risk".

[13] Roi Cohen, Yarin Gal, and Krishnamurthy Dvijotham. 2026. OMNI-LEAK: Multi-Agent Data Leakage Through Implicit Information Channels. *arXiv preprint arXiv:2602.13477* (2026). arXiv:2602.13477 [cs.CR]

[14] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proc. 4th ACM Symposium on Principles of Programming Languages (POPL)*. 238–252. doi:10.1145/512950.512973

[15] Patrick Cousot, Radhia Cousot, Jerôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proc. European Symposium on Programming (ESOP) (LNCS, Vol. 3444)*. 21–30. doi:10.1007/978-3-540-31987-0_3

[16] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, et al. 2025. Malice in Agentland. *arXiv preprint arXiv:2510.05159* (2025). 2% trace poisoning achieves 80% attack success in multi-agent systems.

[17] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252

[18] Roberto di Cosmo and Jérôme Vouillon. 2011. On Software Component Coinstallability. In *Proc. European Symposium on Programming (ESOP) (LNCS, Vol. 6602)*. 202–223. doi:10.1007/978-3-642-19718-5_11

[19] Danny Dolev and Andrew Chi-Chih Yao. 1983. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* 29, 2 (1983), 198–208. doi:10.1109/TIT.1983.1056650

[20] Ruian Duan, Omar Alrawi, Ranjitha Prakash Kasara, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Proc. Network and Distributed Systems Security Symposium (NDSS)*. doi:10.14722/ndss.2021.24383

[21] Ethiack. 2026. CVE-2026-25253: Remote Code Execution in OpenClaw Skill Runtime. Ethiack Vulnerability Disclosure. First CVE for an agentic AI system. Disclosed January 27, 2026.

[22] European Parliament. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union.

[23] Xi Fang, Weiyan Xu, and Shuo Chen. 2026. Agentic AI as Attack Surface: The Viral Agent Loop and Zero-Trust Countermeasures. *arXiv preprint arXiv:2602.19555* (2026). arXiv:2602.19555 [cs.CR]

[24] Donald Gibb et al. 2026. Package Managers à la Carte. *arXiv preprint arXiv:2602.18602* (2026).

[25] Joseph A. Goguen and José Meseguer. 1982. Security Policies and Security Models. In *Proc. IEEE Symposium on Security and Privacy (S&P)*. 11–20. doi:10.1109/SP.1982.10014

[26] Google. 2023. SLSA: Supply-chain Levels for Software Artifacts. https://slsa.dev/. Version 1.0. Four graduated trust levels based on build provenance and integrity guarantees.

[27] Yu Gu, Yiheng Dong, and Hao Sun. 2026. Atomix: Transactional Semantics for Agent Tool Calls. *arXiv preprint arXiv:2602.14849* (2026). arXiv:2602.14849 [cs.SE]

[28] Zhiyuan Gu, Jiaxin Fan, and Xiang Zhang. 2025. Understanding the LLM Supply Chain: A Comprehensive Survey. *arXiv preprint arXiv:2504.20763* (2025). arXiv:2504.20763 [cs.CR]

[29] Richard Harang, Richard Ducott, and Chris Platt. 2025. OWASP MAS Extension: Threat Modeling for Multi-Agent Systems. *arXiv preprint arXiv:2508.09815* (2025). arXiv:2508.09815 [cs.CR]

[30] Jingyu He, Yixin Zhou, and Yang Liu. 2025. Systematic Analysis of Security in the Model Context Protocol. *arXiv preprint arXiv:2508.12538* (2025). arXiv:2508.12538 [cs.CR]

[31] Michael Hicks, Matthew Might, and Sam Tobin-Hochstadt. 2026. LLMbda Calculus: Lambda Calculus with Information-Flow Control for Large Language Models. *arXiv preprint arXiv:2602.20064* (2026). arXiv:2602.20064 [cs.PL] Proves noninterference theorem for LLM information flow.

[32] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives on ChatGPT Plugin Security. *arXiv preprint arXiv:2309.10254* (2023). arXiv:2309.10254 [cs.CR] Early work on LLM platform plugin security.

[33] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. 2018. PySAT: A Python Toolkit for Prototyping with SAT Oracles. In *Proc. International Conference on Theory and Applications of Satisfiability Testing (SAT) (LNCS, Vol. 10929)*. 428–437. doi:10.1007/978-3-319-94144-8_26

[34] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs?. In *Proc. 35th International Conference on Software Engineering (ICSE)*. 672–681.

doi:10.1109/ICSE.2013.6606613
[35] Audun Jøsang. 2001. A Logic for Uncertain Probabilities. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 9, 3 (2001), 279–311. doi:10.1142/S0218488501000831
[36] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. A Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proc. IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP46215.2023.10179304
[37] Zilong Li, Rui Wang, Haichuan Xu, et al. 2026. Malicious Agent Skills in the Wild. *arXiv preprint arXiv:2602.06547* (2026). 98,380 skills scanned across multiple registries; 157 confirmed malicious skills identified.
[38] Linux Foundation. 2022. Sigstore: A New Standard for Signing, Verifying and Protecting Software. https://www.sigstore.dev/.
[39] Zhenyuan Liu, Shuai Wang, and Xiang Li. 2026. MCP-ITP: Automated Framework for Implicit Tool Poisoning in the Model Context Protocol. *arXiv preprint arXiv:2601.07395* (2026). arXiv:2601.07395 [cs.CR]
[40] David R. MacIver and Zac Hatfield-Dodds. 2019. Hypothesis: A New Approach to Property-Based Testing. *Journal of Open Source Software* 4, 43 (2019), 1891. doi:10.21105/joss.01891
[41] Sergio Maffeis, John C. Mitchell, and Ankur Taly. 2010. Object Capabilities and Isolation of Untrusted Web Applications. In *Proc. IEEE Symposium on Security and Privacy (S&P)*. 125–140. doi:10.1109/SP.2010.16
[42] Fabio Mancinelli, Jaap Boender, Roberto di Cosmo, Jerome Vouillon, Berke Durak, Xavier Leroy, and Ralf Treinen. 2006. Managing the Complexity of Large Free and Open Source Package-Based Software Distributions. In *Proc. 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 199–208. doi:10.1109/ASE.2006.49
[43] MCPShield Contributors. 2026. MCPShield: Security Scanning and Lockfile Generation for MCP Servers. https://github.com/nicholaskang/mcpshield. Hash-based mcp.lock.json with SHA-512 integrity checks.
[44] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* Ph. D. Dissertation. Johns Hopkins University. Advisor: Jonathan S. Shapiro.
[45] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. 2003. *Capability Myths Demolished.* Technical Report SRL2003-02. Johns Hopkins University.
[46] Andrew C. Myers and Barbara Liskov. 1997. A Decentralized Model for Information Flow Control. In *Proc. 16th ACM Symposium on Operating Systems Principles (SOSP)*. 129–142. doi:10.1145/268998.266669
[47] National Institute of Standards and Technology. 2023. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. doi:10.6028/NIST.AI.100-1
[48] Thanh Nguyen, Michael Rabin, and Marc Fischer. 2026. Towards Verifiably Safe Tool Use for LLM-Based Agents. *arXiv preprint arXiv:2601.08012* (2026). ICSE-NIER '26. 4-page Alloy-based vision paper; no implementation or evaluation.
[49] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Proc. EuroS&P Workshops*. 23–37. doi:10.1007/978-3-030-52683-2_2
[50] OpenClaw Community. 2026. OpenClaw: Open-Source Agent Framework and Marketplace. https://github.com/openclaw/openclaw. 228,000 GitHub stars as of February 2026.
[51] OWASP. 2023. CycloneDX Software Bill of Materials Standard. https://cyclonedx.org/. Version 1.6.
[52] OWASP. 2024. CycloneDX AI Bill of Materials (AI-BOM). https://cyclonedx.org/capabilities/aibom/.
[53] OWASP. 2024. CycloneDX Specification v1.6. https://cyclonedx.org/specification/overview/. Latest version with AI-BOM support.
[54] Seonghyeon Park, Jihye Kim, Seungwon Lee, et al. 2026. Agent Skills in the Wild. *arXiv preprint arXiv:2601.10338* (2026). 42,447 skills scanned; 26.1% vulnerable; SkillScan tool achieves 86.7% precision.
[55] Dario Pasquini, Shruti Tople, and Mihai Christodorescu. 2025. MCPTox: Benchmark for Tool Poisoning on Real-World MCP Servers. *arXiv preprint arXiv:2508.14925* (2025). arXiv:2508.14925 [cs.CR]
[56] Tom Preston-Werner. 2013. Semantic Versioning 2.0.0. https://semver.org/.
[57] Karthik Raman, Ashwin Swaminathan, and Prateek Reddy. 2025. Securing Agentic AI: A Comprehensive Threat Model and Mitigation Framework. *arXiv preprint arXiv:2504.19956* (2025). arXiv:2504.19956 [cs.CR]
[58] Andrei Sabelfeld and Andrew C. Myers. 2003. Language-Based Information-Flow Security. *IEEE Journal on Selected Areas in Communications* 21, 1 (2003), 5–19. doi:10.1109/JSAC.2002.806121
[59] Rizel Sethi, Julien Abecassis, and Piergiorgio Ladisa. 2025. A Rusty Link in the AI Supply Chain. In *Proc. IEEE Symposium on Security and Privacy Workshops (SPW)*. arXiv:2505.01067 [cs.CR]
[60] Snyk. 2026. Snyk Agent Security Scanner. https://github.com/snyk/agent-scan. Acquired Invariant Labs. LLM-as-judge + heuristic rules.
[61] Chris Tucker, David Shuffelton, Ranjit Jhala, and Sorin Lerner. 2007. OPIUM: Optimal Package Install/Uninstall Manager. In *Proc. 29th International Conference on Software Engineering (ICSE)*. 178–188. doi:10.1109/ICSE.2007.59
[62] Yifan Wang, Zhen Chen, Bo Li, and Dawn Song. 2026. ToolShield: Defending Against Tool-Use Attacks on LLM-Based Agents. *arXiv preprint arXiv:2602.13379* (2026). Heuristic defense achieving -30% ASR reduction.
[63] Yifan Wang, Xiao Liu, and Dawn Song. 2026. MalTool: Benchmarking Malicious Tool Attacks Against LLM Agents. *arXiv preprint arXiv:2602.12194* (2026). 6,487 malicious tools; VirusTotal fails to detect majority.
[64] Edwin Bidwell Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953
[65] Zhengxiang Wu, Yufei Wang, and Zhuowen Li. 2026. NEST: Steganographic Chain-of-Thought Reasoning in Large Language Models. *arXiv preprint arXiv:2602.14095* (2026). arXiv:2602.14095 [cs.CR]
[66] Ziyang Yan, Dong Huang, Yelong Chen, and Robin Jia. 2026. CIBER: Code Interpreter Security—The NL Disguise Attack and Beyond. *arXiv preprint arXiv:2602.19547* (2026). arXiv:2602.19547 [cs.CR]
[67] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. Weak Links in JavaScript Supply Chains: A Large-Scale Empirical Study. In *Proc. ICSE-SEIP*. 235–244.
[68] Wei Zhang, Hao Chen, Jing Li, et al. 2026. SoK: Agentic Skills in the Wild. *arXiv preprint arXiv:2602.20867* (2026). Documents the ClawHavoc campaign: 1,200+ malicious skills infiltrating the OpenClaw marketplace.

About the Author

Varun Pratap Bhardwaj is a Senior Manager and Solution Architect with over 15 years of experience in enterprise technology, spanning cloud architecture, platform engineering, and large-scale system design across Fortune 500 clients in retail, telecommunications, and financial services. He holds dual qualifications in technology and law (LL.B.), providing a distinctive perspective on regulatory compliance for autonomous AI systems, including the EU AI Act, NIST AI RMF, and emerging agent governance frameworks.

His research focuses on formal methods for AI agent safety, building the *Agent Software Engineering* discipline through a suite of peer-reviewed tools and frameworks. Prior published work includes AgentAssert [4], the first design-by-contract framework for autonomous AI agents with runtime behavioral enforcement, and SuperLocalMemory, a privacy-preserving local-first memory system for multi-agent architectures. SkillFortify is the third product in this suite, extending formal guarantees from agent behavior specification to supply chain security.

Contact: varun.pratap.bhardwaj@gmail.com
ORCID: 0009-0002-8726-4289

A Full Proofs

This appendix provides complete, rigorous proofs for all five theorems stated in the main text. We restate each theorem for self-containedness before presenting the proof.

A.1 Theorem 3.6: DY-Skill Maximality

Definition A.1 (DY-Skill Adversary, restated)**.** A *DY-Skill adversary* $\mathcal{A}_{\text{DY-Skill}}$ is a probabilistic polynomial-time (PPT) machine that controls the communication channel between the agent runtime $\mathcal{R}$ and the skill execution environment $\mathcal{E}$. The adversary has the following capabilities:

(i) **Intercept**: $\mathcal{A}$ can observe any message m sent between $\mathcal{R}$ and $\mathcal{E}$.
(ii) **Inject**: $\mathcal{A}$ can inject arbitrary messages m' into the channel.
(iii) **Modify**: $\mathcal{A}$ can replace any message m with $m' \neq m$.
(iv) **Drop**: $\mathcal{A}$ can suppress any message.
(v) **Forge skills**: $\mathcal{A}$ can create skill definitions s' with arbitrary content, including:
- Skill manifest with arbitrary capability declarations C_{declared}

- Executable code implementing capabilities $C_{\text{actual}} \supseteq C_{\text{declared}}$
- Metadata mimicking legitimate skills (name squatting, version manipulation)

(vi) **Compromise registries**: $\mathcal{A}$ can modify skill entries in the registry $\mathcal{P}$, subject to the constraint that $\mathcal{A}$ cannot forge valid cryptographic signatures under keys it does not possess.

Definition A.2 (Classical Dolev-Yao Adversary)**.** A classical Dolev-Yao adversary $\mathcal{A}_{DY}$ operates over a network protocol Π with capabilities (i)–(iv) from the definition above, applied to protocol messages. $\mathcal{A}_{DY}$ additionally possesses the standard Dolev-Yao deduction rules for message construction and decomposition.

Definition A.3 (Simulation)**.** Adversary $\mathcal{A}_1$ *simulates* adversary $\mathcal{A}_2$ (written $\mathcal{A}_1 \succeq \mathcal{A}_2$) if for every attack strategy σ_2 of $\mathcal{A}_2$, there exists a strategy σ_1 of $\mathcal{A}_1$ such that the observable effects on the system are computationally indistinguishable:

$$\forall \sigma_2 \in \mathit{Strat}(\mathcal{A}_2),\ \exists \sigma_1 \in \mathit{Strat}(\mathcal{A}_1):\ \mathit{View}_{\mathcal{R}}(\sigma_1) \approx_c \mathit{View}_{\mathcal{R}}(\sigma_2)$$

where $\mathit{View}_{\mathcal{R}}(\sigma)$ denotes the distribution of observations by the runtime $\mathcal{R}$ under strategy σ, and $\approx_c$ denotes computational indistinguishability.

Theorem A.4 (DY-Skill Maximality, restated). *$\mathcal{A}_{DY\text{-}Skill}$ is maximal among PPT skill adversaries: for any PPT adversary $\mathcal{A}$ operating on agent skill channels, $\mathcal{A}_{DY\text{-}Skill} \succeq \mathcal{A}$. Moreover, $\mathcal{A}_{DY\text{-}Skill} \succeq \mathcal{A}_{DY}$ when the DY-Skill channel is instantiated as a standard network protocol.*

Proof. We prove the two claims separately.

Claim 1: Maximality over skill adversaries.

Let $\mathcal{A}$ be any PPT adversary operating on agent skill channels. We construct a simulation strategy $\sigma_{DY\text{-}Skill}$ for $\mathcal{A}_{DY\text{-}Skill}$ that reproduces any strategy $\sigma_{\mathcal{A}}$ of $\mathcal{A}$.

Any action by $\mathcal{A}$ on the skill channel falls into exactly one of the following categories:

(a) *Channel manipulation* (intercept, inject, modify, drop): These are capabilities (i)–(iv) of $\mathcal{A}_{DY\text{-}Skill}$. The simulation is the identity: $\mathcal{A}_{DY\text{-}Skill}$ performs the same channel operation.

(b) *Skill forgery* (creating a malicious skill definition): This is capability (v). Since $\mathcal{A}$ is PPT, the forged skill s' can be computed in polynomial time. $\mathcal{A}_{DY\text{-}Skill}$ computes the same s' using capability (v) and injects it using capability (ii).

(c) *Registry manipulation* (modifying skill entries): This is capability (vi). $\mathcal{A}_{DY\text{-}Skill}$ applies the same registry modifications, subject to the identical cryptographic constraint.

(d) *Computation on observed data* (e.g., analyzing intercepted messages to craft adaptive attacks): Since both $\mathcal{A}$ and $\mathcal{A}_{DY\text{-}Skill}$ are PPT, any polynomial-time computation that $\mathcal{A}$ performs on observed data can also be performed by $\mathcal{A}_{DY\text{-}Skill}$.

Since every action category of $\mathcal{A}$ maps to a capability of $\mathcal{A}_{DY\text{-}Skill}$, the simulation is complete. For any strategy $\sigma_{\mathcal{A}}$:

$$\mathit{View}_{\mathcal{R}}(\sigma_{DY\text{-}Skill}) = \mathit{View}_{\mathcal{R}}(\sigma_{\mathcal{A}})$$

with equality (not merely computational indistinguishability), since the simulation is exact.

Claim 2: Subsumption of classical DY.

Let σ_{DY} be any strategy of $\mathcal{A}_{DY}$ over a protocol Π. We instantiate the DY-Skill channel as the protocol channel of Π.

Capabilities (i)–(iv) of $\mathcal{A}_{DY}$ map directly to capabilities (i)–(iv) of $\mathcal{A}_{DY\text{-}Skill}$. The Dolev-Yao deduction rules (pairing, projection, encryption, decryption with known keys) are polynomial-time operations, hence executable by the PPT $\mathcal{A}_{DY\text{-}Skill}$.

Capabilities (v) and (vi) of $\mathcal{A}_{DY\text{-}Skill}$ are *additional* capabilities not present in $\mathcal{A}_{DY}$. Therefore, $\mathit{Strat}(\mathcal{A}_{DY}) \subseteq \mathit{Strat}(\mathcal{A}_{DY\text{-}Skill})$, and the simulation is the identity embedding. □

A.2 Theorem 4.9: Analysis Soundness

We prove soundness directly over the capability domain $\mathbb{L}_{cap}^{|C|}$ defined in Section 4.1, which is the domain on which the main-text theorem is stated.

Theorem A.5 (Analysis Soundness, restated). *Let s be a skill with declared capability set $\mathit{Cap}_D(s)$. If the abstract analysis over the capability domain $\mathbb{L}_{cap}^{|C|}$ reports no violations—i.e., $\mathit{Viol}(s) = \emptyset$—then for every concrete execution trace τ of s and every resource access (r, ℓ) performed in τ:*

$$\ell \sqsubseteq \mathit{Cap}_D(s)(r).$$

Proof. We prove the soundness condition $\alpha \circ [\![s]\!] \sqsubseteq [\![s]\!]^\sharp \circ \alpha$ (Proposition 4.7) by verifying that each concrete transfer function from Section 4.3 is soundly over-approximated by the corresponding abstract transfer function in the capability domain. The proof proceeds in two parts: (1) verifying soundness of each per-resource transfer function, and (2) lifting to composite constructs by structural induction.

Part 1: Transfer function soundness (per resource type).

For each resource type $r \in C$, we verify that the abstract transfer function $\tau_r^\sharp$ over-approximates the concrete capability footprint. Let $\alpha_r(S) \in \mathbb{L}_{cap}$ be the abstraction of a set of concrete states S projected onto resource r: the least access level ℓ such that every state in S accesses r at level ℓ or below. We verify $\alpha_r(S) \sqsubseteq \tau_r^\sharp(\alpha_r(S))$ for each transfer function:

(1) **Network (URL reference).** *Pattern:* URL string detected in skill content. *Concrete:* The skill may issue HTTP GET requests, accessing `network` at level READ. *Abstract:* $\tau^\sharp_{\texttt{network}} :=$ READ. *Soundness:* $\alpha_{\texttt{network}}(S) \in \{\text{NONE}, \text{READ}\} \sqsubseteq \text{READ} = \tau^\sharp_{\texttt{network}}$. ✓

(2) **Network (HTTP write method).** *Pattern:* POST, PUT, PATCH, or DELETE detected. *Concrete:* The skill may issue mutating HTTP requests, accessing `network` at level WRITE. *Abstract:* $\tau^\sharp_{\texttt{network}} :=$ WRITE. *Soundness:* Any concrete access is at most WRITE $\sqsubseteq$ WRITE. ✓

(3) **Shell (command invocation).** *Pattern:* Shell command invocation detected. *Concrete:* The skill may execute arbitrary shell commands, accessing `shell` at level WRITE. *Abstract:* $\tau^\sharp_{\texttt{shell}} :=$ WRITE. *Soundness:* Concrete shell access is at most WRITE. The abstract result matches. ✓

(4) **Environment (variable reference).** *Pattern:* Environment variable reference detected. *Concrete:* The skill reads environment variables, accessing `environment` at level READ.

Abstract: $\tau^\sharp_{\texttt{environment}} :=$ READ. *Soundness:* Reading an environment variable is at most READ $\sqsubseteq$ READ. ✓

(5) **Filesystem (read operation).** *Pattern:* File read keyword detected. *Concrete:* The skill reads files, accessing `filesystem` at level READ. *Abstract:* $\tau^\sharp_{\texttt{filesystem}} :=$ READ. *Soundness:* READ $\sqsubseteq$ READ. ✓

(6) **Filesystem (write operation).** *Pattern:* File write keyword detected. *Concrete:* The skill writes files, accessing `filesystem` at level WRITE. *Abstract:* $\tau^\sharp_{\texttt{filesystem}} :=$ WRITE. *Soundness:* WRITE $\sqsubseteq$ WRITE. ✓

(7) **Skill invocation.** *Pattern:* Sub-skill invocation detected. *Concrete:* The skill invokes another skill, accessing `skill_invoke` at level READ (query) or WRITE (mutation). *Abstract:* $\tau^\sharp_{\texttt{skill\_invoke}}$:= WRITE (conservative over-approximation). *Soundness:* Any concrete invocation is at most WRITE $\sqsubseteq$ WRITE. ✓

(8) **Remaining resources** (`clipboard`, `browser`, `database`). Transfer functions follow the same pattern: keyword presence maps to READ or WRITE depending on the detected operation type. Each concrete access is bounded by the abstract inference. ✓

In all cases, the abstract transfer function assigns a capability level $\ell^\sharp$ such that $\alpha_r(S) \sqsubseteq \ell^\sharp$ for every concrete state set S matching the detected pattern. This establishes the local soundness condition: each individual transfer function satisfies the Galois connection requirement.

Part 2: Structural induction over composite constructs.

We lift the per-transfer-function soundness to full skill definitions by structural induction.

Case: Sequential composition $(s_1;\ s_2)$. By the induction hypothesis, both $[\![s_1]\!]^\sharp$ and $[\![s_2]\!]^\sharp$ are sound. The composed abstract semantics is $[\![s_1;\ s_2]\!]^\sharp = [\![s_2]\!]^\sharp \circ [\![s_1]\!]^\sharp$. By monotonicity of both abstract functions and the standard composition lemma for Galois connections [14]:

$$\alpha \circ [\![s_1;\ s_2]\!] \sqsubseteq [\![s_2]\!]^\sharp \circ [\![s_1]\!]^\sharp \circ \alpha = [\![s_1;\ s_2]\!]^\sharp \circ \alpha.$$

Case: Conditional (`if` b `then` s_1 `else` s_2). Concretely, exactly one branch executes. Abstractly, we take the pointwise join: $[\![\texttt{if}\ \cdots]\!]^\sharp(a) = [\![s_1]\!]^\sharp(a) \sqcup [\![s_2]\!]^\sharp(a)$. Since the concrete result equals one branch and the join over-approximates both, soundness follows from the induction hypothesis and the join being an upper bound.

Case: Loop (`while` b `do` s_{body}). The abstract semantics computes the least fixpoint of $\lambda a.\ a_0 \sqcup [\![s_{\text{body}}]\!]^\sharp(a)$. By Tarski's fixpoint theorem and the fact that $\mathbb{L}_{cap}^{|C|}$ has finite height $4^8 = 65{,}536$, the ascending chain stabilizes. The fixpoint over-approximates all concrete loop unrollings by induction on the iteration count.

Conclusion. By structural induction, $\alpha \circ [\![s]\!] \sqsubseteq [\![s]\!]^\sharp \circ \alpha$ for all skill definitions s. Combined with the violation check ($Viol(s) = \emptyset \Rightarrow Cap_I(s) \sqsubseteq Cap_D(s)$), we obtain the theorem: every concrete resource access (r, ℓ) satisfies $\ell \sqsubseteq Cap_I(s)(r) \sqsubseteq Cap_D(s)(r)$. □

A.3 Theorem 5.6: Static Capability Confinement

Definition A.6 (Operation Capability Mapping)**.** Each operation o in a skill's code has a *capability requirement* $cap(o) = (r, \ell) \in C \times \mathbb{L}_{cap}$, mapping it to a resource type and minimum access level:

- cap(`read_file`) = (`filesystem`, READ)
- cap(`write_file`) = (`filesystem`, WRITE)
- cap(`read_env`) = (`environment`, READ)
- cap(`exec`) = (`shell`, WRITE)
- cap(`http_get`) = (`network`, READ)
- cap(`http_post`) = (`network`, WRITE)

The set of all operations syntactically present in skill s is $ops(s)$. The *inferred capability set* is defined resource-wise:

$$Cap_I(s)(r) = \bigsqcup_{\substack{o \in ops(s) \\ cap(o).r = r}} cap(o).\ell$$

with $Cap_I(s)(r) =$ NONE if no operation accesses resource r.

:= Theorem A.7 (Static Capability Confinement, restated). *Let s be a skill with declared capability set $Cap_D(s)$. If $Viol(s) = \emptyset$ (i.e., $Cap_I(s) \sqsubseteq Cap_D(s)$), then for every operation o syntactically reachable in s:*

$$cap(o).\ell \sqsubseteq Cap_D(s)(cap(o).r).$$

Proof. By structural induction on the syntax of skill code. The key invariant is: for every code construct s' in the skill, every operation $o \in ops(s')$ satisfies $cap(o).\ell \sqsubseteq Cap_I(s)(cap(o).r)$. Combined with the hypothesis $Cap_I(s) \sqsubseteq Cap_D(s)$, this yields the theorem by transitivity.

Base case: Single operation o.

The skill consists of a single operation o with $cap(o) = (r, \ell)$. By definition of Cap_I:

$$\ell = cap(o).\ell \sqsubseteq Cap_I(s)(r)$$

since $o \in ops(s)$ and $Cap_I(s)(r)$ is defined as the join over all such operations. By hypothesis $Cap_I(s)(r) \sqsubseteq Cap_D(s)(r)$, so $\ell \sqsubseteq Cap_D(s)(r)$ by transitivity.

Inductive case: Sequential composition $(s_1;\ s_2)$.

We have $ops(s_1;\ s_2) = ops(s_1) \cup ops(s_2)$. By the induction hypothesis, every operation in $ops(s_1)$ and $ops(s_2)$ individually satisfies the bound. Since $Cap_I(s_1;\ s_2)(r) = Cap_I(s_1)(r) \sqcup Cap_I(s_2)(r)$ and the join is the least upper bound, every individual operation capability is below the join:

$$\forall\, o \in ops(s_1;\ s_2):\ cap(o).\ell \sqsubseteq Cap_I(s_1;\ s_2)(cap(o).r) \sqsubseteq Cap_D(s)(cap(o).r).$$

Inductive case: Conditional (`if` b `then` s_1 `else` s_2).

The analysis conservatively includes both branches: $ops(s) = ops(s_1) \cup ops(s_2)$. At runtime, only one branch executes. If branch s_i executes, each operation $o \in ops(s_i)$ satisfies $cap(o).\ell \sqsubseteq Cap_I(s_i)(cap(o).r)$ by the induction hypothesis. Since $ops(s_i) \subseteq ops(s)$, we have $Cap_I(s_i)(r) \sqsubseteq Cap_I(s)(r)$ for all r, and hence $cap(o).\ell \sqsubseteq Cap_D(s)(cap(o).r)$ by transitivity through $Cap_I(s) \sqsubseteq Cap_D(s)$.

Inductive case: Loop (`while` b `do` s_{body}).

The set of operations is $ops(\texttt{while}\ b\ \texttt{do}\ s_{\text{body}}) = ops(s_{\text{body}})$, since the loop body is the only code that executes (repetition does not introduce new syntactic operations). By the induction hypothesis on s_{body}, every $o \in ops(s_{\text{body}})$ satisfies the bound. Each loop iteration executes the same set of operations, so no iteration can exceed the declared capabilities.

Inductive case: Function call ($f(x_1, \ldots, x_k)$).

If f is a built-in operation, it falls under the base case. If f is a user-defined function, its body is a sub-skill with $ops(f) \subseteq ops(s)$ (since f's body is part of the analyzed skill). By the induction hypothesis on f's body, all operations satisfy the bound.

Conclusion. By structural induction, every operation o syntactically reachable during any execution of s satisfies $cap(o).\ell \sqsubseteq Cap_D(s)(cap(o).r)$. The proof relies only on the definition of Cap_I (the pointwise join over operation capabilities) and the hypothesis $Cap_I(s) \sqsubseteq Cap_D(s)$—it does not assume any runtime sandbox mechanism. □

Runtime Confinement (Design Theorem). Theorem 5.7 follows from the object-capability confinement result of Maffeis et al. [41]: in a capability-safe execution environment where (a) there is no ambient authority, (b) capabilities are unforgeable, and (c) the sandbox mediates every action via Equation (7), the only capabilities available to s are those in $Cap_D(s)$. Since $Cap_D(s)$ is the initial endowment and the four Dennis–Van Horn mechanisms [17] can only attenuate (never amplify) authority (Proposition 5.5), the runtime check in Equation (7) suffices to guarantee Equation (10). The formal details follow the proof structure of Maffeis et al. [41, Theorem 4.1], instantiated for the agent skill capability model of Section 5.1.

A.4 Theorem 6.10: Resolution Soundness

Definition A.8 (SAT Encoding)**.** The *SAT encoding* of an ADG $\mathcal{G}$ introduces:

- Boolean variable $x_{s,v}$ for each skill $s \in V$ and version $v \in versions(s)$, where $x_{s,v} = \top$ means version v of skill s is selected.
- *At-most-one* clause for each skill: $\bigwedge_{v \neq v'} (\neg x_{s,v} \vee \neg x_{s,v'})$.
- *At-least-one* clause for each skill: $\bigvee_{v \in versions(s)} x_{s,v}$.
- *Dependency constraint* for each edge $(u, v) \in E$ with constraint $\mu(u, v)$: if $x_{u,v_u} = \top$, then exactly one x_{v,v_v} with $v_v \in sat(\mu(u, v))$ must be true.

where $sat(c)$ denotes the set of versions satisfying constraint c.

Definition A.9 (Valid Resolution)**.** A *valid resolution* of an ADG $\mathcal{G}$ is a function $r : V \to Version$ such that:

(i) $\forall s \in V : r(s) \in versions(s)$ (version exists)
(ii) $\forall (u, v) \in E : r(v) \in sat(\mu(u, v))$ (constraints satisfied)
(iii) The graph (V, E) is acyclic (no circular dependencies)

Theorem A.10 (Resolution Soundness, restated). *The SAT encoding is* equisatisfiable *with the dependency resolution problem: the SAT formula is satisfiable if and only if a valid resolution exists. Moreover, any satisfying assignment σ of the SAT formula yields a valid resolution r_σ, and any valid resolution r yields a satisfying assignment σ_r.*

Proof. We prove both directions of the bijection.

Direction 1: Satisfying assignment ⇒ valid resolution.

Let σ be a satisfying assignment of the SAT formula. Define $r_\sigma(s) = v$ where $\sigma(x_{s,v}) = \top$.

Well-definedness: By the at-least-one clause, at least one $x_{s,v}$ is true for each s. By the at-most-one clause, at most one is true. Hence exactly one version is selected: r_σ is a well-defined function.

Condition (i): Since $x_{s,v}$ only exists for $v \in versions(s)$, we have $r_\sigma(s) \in versions(s)$.

Condition (ii): Let $(u, v) \in E$ with $r_\sigma(u) = v_u$. The dependency constraint clause states: $x_{u,v_u} \Rightarrow \bigvee_{v_v \in sat(\mu(u,v))} x_{v,v_v}$. Since $\sigma(x_{u,v_u}) = \top$ and σ satisfies this clause, there exists $v_v \in sat(\mu(u, v))$ with $\sigma(x_{v,v_v}) = \top$. By the at-most-one clause, $r_\sigma(v) = v_v \in sat(\mu(u, v))$.

Condition (iii): Cycle detection is performed as a preprocessing step (Kahn's algorithm) before SAT encoding. If a cycle exists, the encoding is not generated and the resolution fails with an explicit error. Hence, any resolution produced from the SAT encoding is guaranteed to be over an acyclic graph.

Direction 2: Valid resolution ⇒ satisfying assignment.

Let r be a valid resolution. Define $\sigma_r(x_{s,v}) = \top$ iff $r(s) = v$, and $\sigma_r(x_{s,v}) = \bot$ otherwise.

At-most-one: For each skill s, exactly one version $r(s)$ is selected, so exactly one $x_{s,v}$ is true.

At-least-one: Since $r(s)$ is defined for all $s \in V$, at least one $x_{s,v}$ is true.

Dependency constraints: For each $(u, v) \in E$ with $r(u) = v_u$: $\sigma_r(x_{u,v_u}) = \top$, and by condition (ii) of valid resolution, $r(v) \in sat(\mu(u, v))$. Let $v_v = r(v)$; then $\sigma_r(x_{v,v_v}) = \top$ and $v_v \in sat(\mu(u, v))$. The dependency clause is satisfied.

Bijection. The mappings $\sigma \mapsto r_\sigma$ and $r \mapsto \sigma_r$ are inverses:

- $r_{\sigma_r}(s) = v$ where $\sigma_r(x_{s,v}) = \top$, i.e., where $r(s) = v$. So $r_{\sigma_r} = r$.
- $\sigma_{r_\sigma}(x_{s,v}) = \top$ iff $r_\sigma(s) = v$ iff $\sigma(x_{s,v}) = \top$. So $\sigma_{r_\sigma} = \sigma$.

Hence the encoding is equisatisfiable, and the mappings establish a bijection between satisfying assignments and valid resolutions. □

A.5 Theorem 7.12: Trust Monotonicity

Definition A.11 (Trust Score, restated)**.** The *trust score* of a skill s at time t is:

$$\tau(s, t) = \sum_{i=1}^{k} w_i \cdot \sigma_i(s, t)$$

where:

- k is the number of trust signals.
- $\mathbf{w} = (w_1, \ldots, w_k) \in \Delta^{k-1}$ is a weight vector in the probability simplex ($w_i \geq 0$, $\sum_i w_i = 1$).
- $\sigma_i(s, t) \in [0, 1]$ is the value of trust signal i for skill s at time t.

Definition A.12 (Signal Update)**.** A *signal update* at time $t{+}1$ is a vector $\delta = (\delta_1, \ldots, \delta_k)$ where:

$$\sigma_i(s, t{+}1) = clamp(\sigma_i(s, t) + \delta_i,\ 0,\ 1)$$

An update is *non-negative* if $\delta_i \geq 0$ for all $i \in \{1, \ldots, k\}$.

Theorem A.13 (Trust Monotonicity, restated). *If all signal updates between time t and $t{+}1$ are non-negative (i.e., $\delta_i \geq 0$ for all i) and no temporal decay is applied ($d_i(t) = 1$ for all i, t), then:*

$$\tau(s, t{+}1) \geq \tau(s, t)$$

Proof. We proceed by direct computation.

Step 1: Express the difference.

$$\tau(s,t{+}1) - \tau(s,t) = \sum_{i=1}^{k} w_i \cdot \sigma_i(s,t{+}1) - \sum_{i=1}^{k} w_i \cdot \sigma_i(s,t)$$
$$= \sum_{i=1}^{k} w_i \cdot \big(\sigma_i(s,t{+}1) - \sigma_i(s,t)\big)$$

Step 2: Bound each signal difference.

For each signal i, we have:

$$\sigma_i(s,t{+}1) = clamp(\sigma_i(s,t) + \delta_i,\ 0,\ 1)$$

Since $\delta_i \geq 0$:

$$\sigma_i(s,t) + \delta_i \geq \sigma_i(s,t)$$

The clamp function $clamp(x,0,1) = \max(0, \min(1,x))$ is monotone non-decreasing in x:

- If $\sigma_i(s,t) + \delta_i \leq 1$: $\sigma_i(s,t{+}1) = \sigma_i(s,t) + \delta_i \geq \sigma_i(s,t)$.
- If $\sigma_i(s,t) + \delta_i > 1$: $\sigma_i(s,t{+}1) = 1 \geq \sigma_i(s,t)$ (since $\sigma_i(s,t) \leq 1$).

In both cases:

$$\sigma_i(s,t{+}1) - \sigma_i(s,t) \geq 0$$

Step 3: Combine with non-negative weights.

Since $w_i \geq 0$ (from the simplex constraint) and $\sigma_i(s,t{+}1) - \sigma_i(s,t) \geq 0$ (from Step 2):

$$w_i \cdot \big(\sigma_i(s,t{+}1) - \sigma_i(s,t)\big) \geq 0 \quad \text{for all } i$$

Therefore:

$$\tau(s,t{+}1) - \tau(s,t) = \sum_{i=1}^{k} w_i \cdot \big(\sigma_i(s,t{+}1) - \sigma_i(s,t)\big) \geq 0$$

Step 4: Boundedness.

We additionally verify that $\tau(s,t) \in [0,1]$ for all s,t:

Lower bound: Since $w_i \geq 0$ and $\sigma_i(s,t) \geq 0$ (by the clamp lower bound):

$$\tau(s,t) = \sum_{i=1}^{k} w_i \cdot \sigma_i(s,t) \geq 0$$

Upper bound: Since $\sigma_i(s,t) \leq 1$ (by the clamp upper bound) and $\sum_i w_i = 1$:

$$\tau(s,t) = \sum_{i=1}^{k} w_i \cdot \sigma_i(s,t) \leq \sum_{i=1}^{k} w_i \cdot 1 = 1$$

Conclusion. $\tau(s,t{+}1) \geq \tau(s,t)$ whenever all signal updates are non-negative and no temporal decay is applied. Furthermore, $\tau(s,t) \in [0,1]$ for all s and t. □

Remark A.14 (Effect of Temporal Decay). When temporal decay is active ($d_i(t) < 1$ for some i), the monotonicity property may not hold in general, since the decay factor can reduce trust even in the absence of negative evidence. This is by design: skills that are not actively maintained should see their trust erode over time, reflecting increased risk from unpatched vulnerabilities and compatibility drift. The decay rate and its interaction with positive evidence are configurable parameters in the trust engine.

Table 11: Full attack type distribution in SkillFortifyBench by format.

ID	Attack Type	Claude	MCP	OpenClaw	Total
A1	Data exfiltration (HTTP)	10	10	10	30
A2	Data exfiltration (DNS)	6	6	6	18
A3	Credential theft	10	10	10	30
A4	Arbitrary code execution	10	10	10	30
A5	File system tampering	6	6	6	18
A6	Privilege escalation	6	6	6	18
A7	Steganographic exfiltration	8	8	8	24
A8	Prompt injection	8	8	8	24
A9	Reverse shell	8	8	8	24
A10	Cryptocurrency mining	4	4	4	12
A11	Typosquatting	4	2	2	8
A12	Dependency confusion	2	4	2	8
A13	Encoded/obfuscated payloads	8	8	10	26
	Total malicious	**90**	**90**	**90**	**270**
	Benign	90	90	90	270
	Grand total	**180**	**180**	**180**	**540**

B SkillFortifyBench Details

This appendix provides the full specification of SkillFortifyBench, including the attack type distribution across formats, example skills, and the labeling methodology.

B.1 Attack Type Distribution

Table 11 shows the complete distribution of attack types across the three skill formats. The benchmark contains 540 skills total: 270 malicious (50%) and 270 benign (50%). Each format (Claude, MCP, OpenClaw) contributes 180 skills (90 malicious + 90 benign).

B.2 Example Skills

We present representative examples of malicious and benign skills in each format. All examples are simplified for exposition; full skills in SkillFortifyBench contain additional realistic metadata.

Listing 1: Malicious Claude skill exfiltrating environment variables.

B.2.1 Malicious Claude Code Skill (A1 — Data Exfiltration via HTTP).

```
---
name: env-helper
description: Helps manage environment variables
capabilities:
  - read_env
---

# Environment Helper

This skill helps manage environment variables.

## Implementation

```python
import os
import requests

def get_env_summary():
    env_data = dict(os.environ)
    # Exfiltration disguised as "analytics"
    requests.post(
        "https://evil.example.com/collect",
```
```

```
        json={"telemetry": env_data}
    )
    return "Environment loaded."
```

SkillFortify detects this skill through two findings: (1) pattern match on `os.environ` combined with `requests.post` to an external URL (A1 pattern), and (2) information flow analysis tracing data from the *env* source through `env_data` to the *net* sink at `requests.post`.

Listing 2: Malicious MCP server with embedded code execution.

B.2.2 Malicious MCP Server Configuration (A4 — Arbitrary Code Execution).

```
{
  "mcpServers": {
    "data-processor": {
      "command": "python",
      "args": ["-c",
        "import subprocess; subprocess.Popen(
         ['sh', '-c',
          'curl evil.example.com/payload|sh']"],
      "env": {
        "PATH": "/usr/bin:/usr/local/bin"
      }
    }
  }
}
```

SkillFortify detects the `subprocess.Popen` with shell invocation piped through `curl`, matching the A4 arbitrary code execution pattern. The command passes inline Python via `-c`, a pattern commonly observed in ClawHavoc [68] payloads.

Listing 3: Benign OpenClaw skill for JSON formatting.

B.2.3 Benign OpenClaw Skill.

```
name: json-formatter
version: 1.2.0
description: Format and validate JSON files
author: verified-publisher
actions:
  - name: format
    description: Pretty-print a JSON file
    input:
      type: string
      description: Path to JSON file
    output:
      type: string
      description: Formatted JSON content
permissions:
  - read_local
```

This skill declares only `read_local` permission, consistent with its stated purpose. SkillFortify correctly classifies it as benign: no malicious patterns match, no information flow violations are detected, and the inferred capabilities ({ReadLocal}) are confined within declared capabilities.

B.3 Labeling Methodology

SkillFortifyBench uses *constructive ground truth*: skills are generated by a deterministic benchmark generator that embeds specific, known attack patterns into malicious skills and generates clean implementations for benign skills. This approach has three advantages over post-hoc labeling:

(1) **Perfect ground truth.** Each skill's label is determined at generation time, eliminating inter-annotator disagreement and labeling errors.

(2) **Reproducibility.** The generator uses seeded pseudo-random number generators (Python `random.seed(42)`) for all stochastic choices (variable names, formatting variations, red-herring code), ensuring byte-identical benchmark regeneration.

(3) **Controlled complexity.** Each malicious skill contains exactly one primary attack type, enabling precise per-type evaluation. Real-world malware often combines multiple techniques; single-type benchmarking provides cleaner attribution of detection success and failure.

Attack pattern sources. The 13 attack types are derived from three verified sources:

- **ClawHavoc campaign** (arXiv:2602.20867, Feb 24, 2026) [68]: 1,200+ malicious skills documented with 7 skill design patterns. Contributes patterns for A1–A6 and A13.
- **MalTool dataset** (arXiv:2602.12194, Feb 12, 2026) [63]: 6,487 malicious tools catalogued. Contributes patterns for A7–A10.
- **CVE-2026-25253 advisory** [21]: Remote code execution via skill installation. Contributes the A4 variant specific to the exploitation vector.
- **Literature survey** ("Agent Skills in the Wild," arXiv:2601.10338) [54]: 14 vulnerability patterns across 42,447 skills. Contributes patterns for A11 and A12. Corroborated by a second large-scale study scanning 98,380 skills with 157 confirmed malicious entries [37].

Benign skill sources. Benign skills are modeled after legitimate skill categories observed in production skill registries: file management (read, write, format), API integration (REST client wrappers), data transformation (JSON, CSV, XML processing), development tooling (linting, testing helpers), and system information (uptime, disk usage, version checks). Each benign skill is designed to use only the capabilities it declares, with no extraneous network access, process execution, or environment variable exposure.

C Lockfile Schema

This appendix specifies the complete `skill-lock.json` schema, field descriptions, and an example lockfile produced by `skillfortify lock`.

C.1 Schema Specification

The lockfile is a JSON document conforming to the following schema. We present it in JSON Schema (Draft 2020-12) notation.

Listing 4: `skill-lock.json` schema (JSON Schema Draft 2020-12).

```
{
  "$schema":
    "https://json-schema.org/draft/2020-12/schema",
  "title": "SkillFortify Skill Lockfile",
  "type": "object",
  "required": [
    "lockfile_version", "generated_at",
    "generated_by", "skills", "integrity"
  ],
  "properties": {
    "lockfile_version": {
      "type": "string",
      "const": "1.0.0",
```

```
      "description":
        "Schema version of the lockfile."
    },
    "generated_at": {
      "type": "string",
      "format": "date-time",
      "description":
        "ISO 8601 timestamp of generation."
    },
    "generated_by": {
      "type": "string",
      "description": "Tool name and version."
    },
    "skills": {
      "type": "object",
      "additionalProperties": {
        "$ref": "#/$defs/LockedSkill"
      },
      "description":
        "Map: skill name -> locked resolution."
    },
    "integrity": {
      "type": "string",
      "description":
        "SHA-256 hash of canonical skills JSON."
    }
  },
  "$defs": {
    "LockedSkill": {
      "type": "object",
      "required": [
        "version", "format", "hash",
        "capabilities", "dependencies"
      ],
      "properties": {
        "version": {
          "type": "string",
          "description":
            "Resolved semantic version."
        },
        "format": {
          "type": "string",
          "enum": ["claude", "mcp", "openclaw"],
          "description": "Skill format type."
        },
        "hash": {
          "type": "string",
          "pattern": "^sha256:[a-f0-9]{64}$",
          "description": "SHA-256 content hash."
        },
        "capabilities": {
          "type": "array",
          "items": { "type": "string" },
          "description":
            "Declared capability set."
        },
        "trust_score": {
          "type": "number",
          "minimum": 0.0,
          "maximum": 1.0,
          "description":
            "Trust score at lock time."
        },
        "dependencies": {
          "type": "object",
          "additionalProperties": {
            "type": "string"
          },
          "description":
            "Map: dep name -> resolved version."
        },
        "analysis": {
          "$ref": "#/$defs/AnalysisResult"
        }
      }
    },
    "AnalysisResult": {
      "type": "object",
      "properties": {
        "status": {
          "type": "string",
          "enum": [
            "clean", "warning", "critical"
          ],
          "description":
            "Overall analysis status."
        },
        "findings_count": {
          "type": "integer",
          "minimum": 0,
          "description":
            "Number of findings."
        },
        "max_severity": {
          "type": "string",
          "enum": [
            "NONE", "LOW", "MEDIUM",
            "HIGH", "CRITICAL"
          ],
          "description":
            "Highest finding severity."
        },
        "analyzed_at": {
          "type": "string",
          "format": "date-time",
          "description":
            "Timestamp of last analysis."
        }
      }
    }
  }
}
```

C.2 Field Descriptions

Table 12 provides detailed descriptions of all top-level and nested fields.

C.3 Example Lockfile

Listing 5 shows an example `skill-lock.json` for a three-skill agent configuration.

Listing 5: Example `skill-lock.json` for a three-skill configuration.

```
{
  "lockfile_version": "1.0.0",
  "generated_at": "2026-02-26T14:30:00Z",
  "generated_by": "skillfortify 0.1.0",
  "skills": {
    "api-client": {
      "version": "3.0.1",
      "format": "mcp",
      "hash": "sha256:c5d6e7f8a9b0c1d2e3f4
        a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1
        f2a3b4c5d6",
      "capabilities": [
        "read_env", "net_access"
      ],
      "trust_score": 0.68,
      "dependencies": {},
      "analysis": {
        "status": "warning",
        "findings_count": 1,
        "max_severity": "LOW",
        "analyzed_at":
          "2026-02-26T14:29:59Z"
      }
    },
    "file-manager": {
      "version": "2.1.0",
      "format": "claude",
```

Table 12: Lockfile field descriptions.

Field	Type	Description
`lockfile_version`	string	Schema version for forward compatibility. Current: `"1.0.0"`.
`generated_at`	datetime	UTC timestamp when the lockfile was generated. Enables audit trail tracking.
`generated_by`	string	Identifies the tool and version (e.g., `"skillfortify 0.1.0"`).
`skills`	object	Maps skill names (keys) to their locked resolution objects. Keys are sorted lexicographically for deterministic serialization.
`integrity`	string	SHA-256 hash computed over the canonical JSON serialization of the `skills` object. Detects tampering with the lockfile after generation.
Per-skill fields (within each entry in `skills`*):*		
`version`	string	The resolved semantic version (e.g., `"1.2.0"`). Determined by the SAT-based resolver.
`format`	string	Skill format: `"claude"`, `"mcp"`, or `"openclaw"`.
`hash`	string	SHA-256 content hash of the skill file at lock time, prefixed with `"sha256:"`. Enables integrity verification on subsequent installs.
`capabilities`	array	The declared capability set from the skill manifest.
`trust_score`	number	Trust score $\tau(s, t)$ at lock generation time. Range: $[0, 1]$.
`dependencies`	object	Maps dependency skill names to their resolved versions. Empty object `{}` if no dependencies.
`analysis.status`	string	Overall analysis result: `"clean"` (no findings), `"warning"` (low-severity findings), or `"critical"` (medium+ severity findings).
`analysis.findings_count`	integer	Total number of analysis findings.
`analysis.max_severity`	string	Highest severity level among all findings.
`analysis.analyzed_at`	datetime	Timestamp of the analysis run.

```
    "hash": "sha256:a3f2b8c9d4e5f6a7b8c9
      d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6
      c7d8e9f0a1",
    "capabilities": [
      "read_local", "write_local"
    ],
    "trust_score": 0.85,
    "dependencies": {},
    "analysis": {
      "status": "clean",
      "findings_count": 0,
      "max_severity": "NONE",
      "analyzed_at":
        "2026-02-26T14:29:58Z"
    }
  },
  "json-formatter": {
    "version": "1.2.0",
    "format": "openclaw",
    "hash": "sha256:b4c5d6e7f8a9b0c1d2e3
      f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0
      e1f2a3b4c5",
    "capabilities": [
      "read_local"
    ],
    "trust_score": 0.72,
    "dependencies": {
      "file-manager": ">=2.0.0"
    },
    "analysis": {
      "status": "clean",
      "findings_count": 0,
      "max_severity": "NONE",
      "analyzed_at":
        "2026-02-26T14:29:59Z"
    }
  }
},
"integrity": "sha256:e7f8a9b0c1d2e3f4a5b6
  c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4
  c5d6e7f8a9"
}
```

Key properties of the example. The lockfile demonstrates several features of the schema:

(1) **Deterministic ordering.** Skills are sorted alphabetically by name (`api-client`, `file-manager`, `json-formatter`). Repeated invocations of `skillfortify lock` on the same configuration produce byte-identical output (verified in Experiment E6, Section 9.7).

(2) **Dependency resolution.** The `json-formatter` skill declares a dependency on `file-manager` with constraint `>=2.0.0`. The resolver selected version `2.1.0`, which satisfies the constraint.

(3) **Trust scores at lock time.** Each skill records its trust score at the moment of lockfile generation. This enables temporal comparison: if a future `skillfortify lock` produces a lower trust score for the same skill version, it signals degraded confidence (e.g., due to newly discovered vulnerabilities or absence of maintenance activity).

(4) **Analysis status.** The `api-client` skill has a `"warning"` status with one low-severity finding. This does not prevent lockfile generation (only `"critical"` status blocks locking by default) but is recorded for audit purposes.

(5) **Integrity hash.** The top-level `integrity` field is a SHA-256 hash of the canonical JSON serialization of the entire `skills` object. Any modification to skill entries, versions, or hashes invalidates the integrity hash, providing tamper detection.

Canonical serialization. Deterministic JSON output is achieved through: (i) sorted dictionary keys at all nesting levels, (ii) consistent whitespace formatting (2-space indentation), (iii) no trailing commas, and (iv) UTF-8 encoding without byte-order mark. These conventions ensure that the SHA-256 integrity hash is stable across platforms and Python versions.